Limbajul PL/SQL

Inroducere; Variabile, expresii si operatori

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Limbajul PL/SQL

- PL/SQL (Procedural Language/SQL) este o extensie procedurală a limbajului SQL pentru SGBD Oracle (extensie proprietară).
- Apariţia sa este datorată faptului că limbajul SQL este un limbaj de cereri şi nu un limbaj de programare.
- Unitatea de execuţie este **blocul** care conţine atât cereri SQL cât şi instrucţiuni PL/SQL. Ele sunt executate astfel:
  - Cererile SQL sunt trimise pentru a fi executate de serverul Oracle. Rezultatele acestora pot fi folosite apoi în instrucţiunile PL/SQL.
  - Instrucţiunile PL/SQL sunt executate de **Executorul de Instrucţiuni Procedurale** (Procedural Statement Executor) aflat în **Motorul PL/SQL** (PL/SQL Engine).

Limbajul PL/SQL

- Motorul PL/SQL este prezent în:
  - Serverul Oracle
  - Unelte Oracle care conţin propriul lor motor PL/SQL. Ele nu folosesc motorul PL/SQL integrat în serverul Oracle decât pentru porţiunile de bloc reprezentând cereri SQL, minimizând astfel încărcarea serverului.

Limbajul PL/SQL

- PL/SQL este folosit pentru a scrie:
  - Blocuri anonime (Anonymous blocks)
  - Proceduri şi funcţii stocate (Stored procedures/functions)
  - Proceduri şi funcţii folosite în aplicaţii (Application procedures/functions)
  - Declanşatori ataşaţi bazei de date (Database triggers)
  - Declanşatori folosiţi în aplicaţii (Application triggers)
  - Pachete (Packaged procedures)

Avantajele folosirii PL/SQL

- Integrare
- Îmbunătăţirea performanţelor
- Dezvoltare modulară a programelor
- Portabilitate
- Este procedural

Avantajele folosirii PL/SQL

Integrare:

- Prezenţa motorului PL/SQL atât în serverul Oracle cât şi în uneltele Oracle
- Bibliotecile comune (shared libraries) cu proceduri şi funcţii scrise în PL/SQL pot fi utilizate de toate aplicaţiile dezvoltate cu Developer/2000 şi pot fi accesate local sau la distanţă.

Avantajele folosirii PL/SQL

- **Integrare:**

- Tipurile de date din SQL pot fi folosite şi în PL/SQL, dar există anumite diferenţe privind dimensiunile maxime pentru unele dintre ele. Aceste diferenţe sunt discutate în capitolele următoare
- În cazul uneltelor Oracle având încorporat motor PL/SQL, executorul de instrucţiuni procedurale procesează date locale aplicaţiei (client) reducând încărcarea serverului şi necesarul de memorie pentru acesta.

Avantajele folosirii PL/SQL

- **Îmbunătăţirea performanţelor:**

- Un bloc care poate conţine mai multe instrucţiuni SQL este trimis o singură dată serverului reducându-se traficul pe reţea.
- Mărirea performanţelor uneltelor Oracle prin adăugarea facilităţilor procedurale ale PL/SQL.
- Procedurile şi funcţiile care fac parte din aplicaţii dezvoltate cu Developer/2000 sunt diferite de cele stocate în baza de date deşi au aceeaşi structură. În acelaşi timp aceste aplicaţii pot folosi subprogramele stocate în baza de date gestionată de serverul Oracle.

Avantajele folosirii PL/SQL

Dezvoltare modulară a programelor:

- Un program PL/SQL (PL/SQL unit) este format din unul sau mai multe blocuri – proceduri, funcţii şi blocuri anonime. Un bloc poate conţine în interiorul său alte blocuri.
- Un program complex poate fi divizat în module logice distincte implementate prin blocuri.
- Codul reutilizabil PL/SQL poate fi memorat în librării comune în cazul aplicaţiilor Developer/2000 sau stocat pe server, în care caz poate fi folosit de orice aplicaţie care interacţionează cu baza de date Oracle.

Avantajele folosirii PL/SQL

Portabilitate:

- Este acelaşi pentru orice platformă Oracle, indiferent de sistemul de operare sau platforma hardware folosită.
- Programele scrise în PL/SQL pot fi folosite în diverse medii (server, aplicaţii).

Avantajele folosirii PL/SQL

Limbaj procedural:

- Se pot defini variabile, cursori, excepţii (tratarea erorilor). Variabilele pot fi scalare, referinţă, compuse sau de tip obiect mare (large object - LOB). Acestea pot fi definite şi dinamic pe baza structurii tabelelor din baza de date şi a coloanelor acestora.
- Există instrucţiuni de decizie, cicluri, parcurgere (pentru liniile unui rezultat de cerere SQL de tip SELECT).

Avantajele folosirii PL/SQL

Limbaj procedural:

- Permite tratarea erorilor apărute în execuţia cererilor SQL de către serverul Oracle (erorile se numesc în acest context **excepţii**) prin rutine asociate diverselor tipuri de erori care pot apare.
- Se pot declara de către utilizator condiţii suplimentare de eroare şi asocia rutine de tratare.

CE PUTEM SCRIE IN PL/SQL?

- Blocuri anonime (Anonymous blocks)
- Proceduri şi funcţii stocate (Stored procedures/functions)
- Proceduri şi funcţii folosite în aplicaţii (Application procedures/functions)
- Declanşatori ataşaţi bazei de date (Database triggers)
- Declanşatori folosiţi în aplicaţii (Application triggers)
- Pachete (Packaged procedures)

CUM ARATA UN BLOC?

- Un bloc anonim are urmatoarea structura:

```
DECLARE      -- opţional
declaraţii de variabile, cursori, excepţii definite de utilizator
BEGIN        -- obligatoriu
cereri SQL
instrucţiuni PL/SQL
EXCEPTION    -- opţional
acţiuni executate în caz de ridicare excepţii
END;         -- obligatoriu
```

Structura unui bloc

Între DECLARE şi BEGIN:

- Este zona de declaraţii a blocului. Sunt declarate în principal variabile, cursori, excepţii utilizator necesare blocului, dar şi alte elemente (tipuri, proceduri, funcţii, etc.).
- Această parte este opţională, un bloc putând începe direct cu cuvântul cheie BEGIN (care este obligatoriu).
- Tipurile de date din SQL pot fi folosite (cu unele diferenţe) şi în PL/SQL.

Structura unui bloc

Între BEGIN şi EXCEPTION:

- Corpul blocului, format din cereri SQL şi instrucţiuni PL/SQL care descriu procesarea datelor în cadrul acestuia.
- Cuvântul cheie EXCEPTION poate lipsi, în care caz blocul se termină cu END.

Structura unui bloc

Între EXCEPTION şi END:

- Acţiuni executate în caz de excepţie (în principal tratarea erorilor).
- Este o parte opţională a unui bloc (dar cuvântul cheie END care marchează sfârşit de bloc este obligatoriu).

Exemplu

```
DECLARE
v_nume    VARCHAR2(10);
BEGIN
SELECT NUME
INTO v_nume
FROM SPEC
WHERE CODS=11;
DBMS_OUTPUT.PUT_LINE('Specializarea: '|| v_nume);
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('A aparut o exceptie');
END;
```

Executia blocului

În cazul execuţiei un bloc din SQL*Plus:

- Fiecare cerere SQL şi instrucţiune PL/SQL se încheie cu punct şi virgulă (;)
- DECLARE, BEGIN şi EXCEPTION nu se termină cu punct şi virgulă
- După END se pune punct şi virgulă
- Terminarea editarii blocului se face cu punct (**.**)
- Rularea unui bloc se face cu **/** sau **r** (run) la promptul SQL>. În sistemul de operare Windows rularea se poate comanda şi prin File → Run.

Executia blocului

În cazul execuţiei un bloc din SQL*Plus:

- Ultimul bloc executat poate fi editat. În acest caz marcajul de terminare bloc este ca şi în cazul cererilor SQL un slash (**/**) aflat singur pe ultima linie a fişierului.
- Editarea se poate iniţia prin comanda **ed** la promptul SQL> sau pe calea:

Edit → *Editor* → *Invoke Editor*

Executia blocului

În cazul execuţiei un bloc din SQL*Plus:

- În caz de rulare cu succes, după eventualele mesaje tipărite de bloc apare:

  `PL/SQL procedure successfully completed`

- În caz de rulare cu eroare netratată, se va afişa codul şi mesajul de eroare.

Executia blocului

- În exemplul de mai sus a fost folosită pentru afişarea mesajului 'A aparut o exceptie' procedura DBMS_OUTPUT.PUT_LINE.
- Aceasta face parte din pachetul DBMS_OUTPUT pus la dispoziţie de sistemul Oracle şi va fi utilizată şi în capitolele următoare. Sintaxa este:

```
DBMS_OUTPUT.PUT_LINE(expresie)
```

Executia blocului

- Procedura evaluează valoarea expresiei, o converteşte la şir de caractere şi o tipăreşte.
- Cum implicit SQL*Plus nu afişează mesajele tipărite în cadrul blocului, pentru a le putea vedea opţiunea SQL*PLUS ***serveroutput*** trebuie trecută pe ON prin comanda:
- **SQL> SET SERVEROUTPUT ON**

Variabile în PL/SQL

- Un bloc PL/SQL poate avea propriile sale variabile.
- În continuare sunt prezentate elementele de bază referitoare la declararea şi folosirea acestora.

Categorii de variabile

- Variabile PL/SQL
- Variabile externe

Variabile PL/SQL

Scalare:

- Aceste variabile pot conţine o singură valoare. Tipurile principale corespund celor care se pot asocia coloanelor unei tabele Oracle.

Compuse:

- Conţin mai multe valori, de exemplu o înregistrare (linie) din rezultatul unei cereri SQL

Variabile PL/SQL

Referinţă:

- Conţin pointeri (referinţe) către alte elemente de program. Nu sunt tratate în acest curs (cu excepţia tipului REF CURSOR)

LOB (obiecte mari):

- Acestea conţin valori numite locatori (locators) care specifică locaţia unor obiecte mari (imagini de exemplu).

Variabile externe

- Variabile care provin din limbajul gazdă în care este scrisă aplicaţia (C, PHP, etc).
- Câmpuri ale unei forme în Oracle Forms
- Variabile SQL*Plus

Declararea variabilelor

- Sintaxa unei declaraţii de variabile este:

```
Identificator [CONSTANT]
tipdedate [NOT NULL] [:= |
DEFAULT expresie]
```

Unde **Identificator :**

- Numele variabilei (maxim 30 caractere).
- Se aplică aceleaşi reguli ca la orice alt obiect SQL (tabele, coloane, etc).

Declararea variabilelor

- Două obiecte pot avea acelaşi nume doar dacă sunt declarate în blocuri diferite. În acest caz în fiecare bloc poate fi folosit doar cel declarat local.
- Este de dorit ca variabilele să nu aibă aceleaşi nume cu coloanele sau tabelele din baza de date. În caz contrar Oracle consideră numele ca fiind cel al coloanei sau tabelei.

Declararea variabilelor

[CONSTANT]

- Variabila nu-şi poate schimba valoarea. Constantele trebuiesc iniţializate la declarare.

tipdedate

- Tipul variabilei (un tip scalar, compus, referinţă sau LOB).

[NOT NULL]

- Variabila nu poate fi nulă. Aceste variabile trebuiesc iniţializate la declarare.

Declararea variabilelor

[:= expresie]

- Iniţializare cu o expresie. Variabilele neiniţializate conţin iniţial valoarea NULL.

[DEFAULT expresie]

- Valoare implicită dată de o expresie (iniţializează variabila).

Exemplu

```
DECLARE
v_nume  VARCHAR2(10)  := 'ION';
v_datanasterii     DATE;
v_cods  NUMBER(2) NOT NULL := 21;
v_suma  CONSTANT NUMBER := 100;
.....
```

Asignarea variabilelor

- Instrucţiunea de atribuire este **:=** (ca în limbajul Pascal).
- Caracterul **=** este folosit în PL/SQL ca operator de comparaţie (egal cu).
- Sintaxa atribuirii este:

```
Identificator := expresie;
```

Exemple

```
v_nume := 'ION';
v_datanasterii := '12-APR-89';
v_data2 := to_date('14-04-79', 'DD-MM-YY');
v_numar := v_n1 + v_n2 * 1230;
```

Tipuri de date folosite

- Tipuri numerice scalare
- Tipuri scalare caracter/binare
- Tipuri LOB (Large Objects)
- Tipul BOOLEAN
- Tipuri Data, Timp si Interval
- Variabile definite cu %TYPE
- Variabile externe

Tipuri numerice scalare

- **NUMBER[(număr de cifre, număr de zecimale)]:** Număr întreg sau real. Fără parametri sau cu 2 parametri: număr real. Cu un parametru: număr întreg. Magnitudine: 10^{-130} - 10^{125} (+/-).
- Subtipuri ale acestui tip sunt:
  - DEC, DECIMAL, NUMERIC – numere în virgulă fixă cu 38 de cifre semnificative.
  - DOUBLE PRECISION, FLOAT: numere în virgulă mobilă cu 38 de cifre semnificative.
  - REAL: numere în virgulă fixă cu 18 cifre semnificative.
  - INTEGER, INT, SMALLINT: numere întregi cu maxim 38 de cifre semnificative.

Tipuri numerice scalare

- **BINARY_INTEGER:** Întregi între -2^{31} şi 2^{31}.
  Subtipuri ale acestuia sunt:
  - NATURAL şi POSITIVE pentru întregi non-negativi sau pozitivi.
  - NATURALN şi POSITIVEN ca mai sus dar nu pot avea valori nule
  - SIGNTYPE: doar valorile -1, 0 şi 1.

Tipuri numerice scalare

- **PLS_INTEGER:** Întregi între -2^{31} şi 2^{31}. E asemănător cu tipul anterior dar operaţiile sunt mai rapide.
- În plus, în caz de depăşire se ridică o excepţie (la BINARY_INTEGER şi NUMBER nu se întâmplă asta).
- Este recomandat în aplicaţii noi în loc de BINARY_INTEGER.

Tipuri scalare caracter/binare

CHAR[(lungime_maximă [CHAR | BYTE])]:

- Şir de caractere de lungime fixă (maxim 32737 octeţi, implicit 1). Opţional CHAR | BYTE spune în ce este exprimată lungimea maximă (un caracter putând fi memorat pe mai mulţi octeţi în funcţie de setul de caractere folosit).
- Un subtip al acestui tip este CHARACTER, identic cu tipul de bază (introdus pentru compatibilitatea cu alte sisteme).
- **Notă:** acest tip diferă de tipul CHAR pentru coloane care poate avea maxim 2000 de octeţi.

Tipuri scalare caracter/binare

VARCHAR2(lungime_maximă [CHAR | BYTE]):

- Şir de caractere de lungime variabilă, maxim 32737 octeţi. Aceleaşi observaţii ca mai sus în legatură cu CHAR | BYTE.
- Subtipuri ale acestui tip sunt: STRING şi VARCHAR, identice cu tipul de bază (introduse pentru compatibilitatea cu alte sisteme).
- **Notă:** acest tip diferă de tipul VARCHAR2 pentru coloane care poate avea maxim 4000 de octeţi.

Tipuri scalare caracter/binare

LONG

- Similar cu VARCHAR2 dar lungimea maximă este de 32760 de octeţi. Începând cu Oracle 9i acest tip este compatibil cu CLOB şi se recomandă folosirea acestuia din urmă.
- **Notă:** acest tip diferă de tipul LONG pentru coloane care poate avea maxim 2^{31} octeţi (2 GB).

Tipuri scalare caracter/binare

LONG RAW:

- Similar cu LONG dar se pot stoca şi date binare (nu doar caractere). În plus, PL/SQL nu interpretează date de acest tip.
- Începând cu Oracle 9i acest tip este compatibil cu BLOB şi se recomandă folosirea acestuia din urmă.
- **Notă:** acest tip diferă de tipul LONG RAW pentru coloane care poate avea maxim 2^{31} octeţi.

Tipuri scalare caracter/binare

RAW(dimensiune_maximă)

- Similar cu VARCHAR2 conţine date binare. Dimensiunea maximă este de 32767 octeţi. PL/SQL nu interpretează astfel de date.
- **Notă:** acest tip diferă de tipul LONG RAW pentru coloane care poate avea maxim 2000 de octeţi.

Tipuri scalare caracter/binare

ROWID

- Poate stoca un identificator fizic de linie într-o tabelă. Pentru conversia la/de la şir de caractere (18 caractere) se pot folosi funcţiile SQL ROWIDTOCHAR respectiv CHARTOROWID.

UROWID

- (Universal ROWID) Poate stoca un identificator logic de linie într-o tabelă, indexată sau nu precum şi un identificator de linie extern (non-Oracle). Nu este necesară folosirea funcţiilor de conversie la/de la şir de caractere (conversie automată).

Tipuri scalare caracter/binare

NCHAR(dimensiune_maximă)

- Echivalent cu CHAR dar pentru a stoca date în setul de caractere naţional specificat la crearea bazei (un caracter pe 2-3 octeţi).

NVARCHAR2(dimensiune maximă)

- Echivalent cu VARCHAR2 dar pentru a stoca date în setul de caractere naţional specificat la crearea bazei (un caracter pe 2-3 octeţi).

Tipuri LOB

- Aceste tipuri permit stocarea de date nestructurate (text, imagini, video, audio) de dimensiune maximă 4 Gbytes.
- Este posibil accesul direct la conţinut (la LONG doar acces secvenţial).
- Obiectele de acest tip conţin de fapt referinţe (locatori) pentru date, datele propriu-zise fiind stocate fie în baza de date fie în fişiere externe din sistemul de operare.
- Sunt de următoarele tipuri:

Tipuri LOB

BFILE:

- Date binare (până la 4GB) stocate în fişiere externe, din sistemul de operare. Sunt date read/only (nu se pot modifica). Nu participă la tranzacţii şi nici la restaurarea bazei de date (ele nu sunt stocate în aceasta) şi nu pot fi replicate.

Tipuri LOB

BLOB:

- Pentru date binare de dimensiuni mari. Aparţin bazei de date, participă la tranzacţii şi se pot restaura cu baza de date, sunt replicabile. Dimensiunea maximă este de 4GB.

CLOB:

- Similar cu BLOB dar pentru date tip caracter şi nu binare. Aceleaşi caracteristici ca mai sus.

NCLOB:

- Similar cu CBLOB dar pentru şiruri de caractere din setul naţional setat la crearea bazei de date.

Tipul BOOLEAN

- Se pot stoca valori booleene: TRUE, FALSE şi NULL. Sunt permişi doar operatorii booleeni.
- **Notă:** acest tip nu există ca tip pentru coloanele unei tabele.

Tipuri Data, Timp, Interval

DATE:

- Dată calendaristică (zi, lună, an, oră, minut, secundă). Se pot folosi operatorii + număr şi - număr pentru a incrementa şi decrementa cu un număr de zile o dată.
- Diferenţa între două date dă numărul de zile dintre acestea (real, cu zecimale în cazul în care ora-minutul-secunda diferă).

Tipuri Data, Timp, Interval

TIMESTAMP[(precizie)]

- Extinde tipul DATE. Conţine dată calendaristică (zi, lună, an, oră, minut, secundă şi fracţiuni de secundă).
- Precizia (între 0 şi 9) specifică numărul de zecimale pentru fracţiunile de secundă. Valoarea implicită a preciziei este 6.

Exemplu:

```
DECLARE
t TIMESTAMP(4);
BEGIN
t := TIMESTAMP '1988-12-26 15:52:33.1592';
END;
```

Tipuri Data, Timp, Interval

TIMESTAMP [(precizie)] WITH TIME ZONE

- Extinde tipul TIMESTAMP conţinând şi o diferenţă între ora locală şi ora universală (GMT).

Exemplu:

```
DECLARE
t TIMESTAMP(4) WITH TIME ZONE;
BEGIN
t := TIMESTAMP '1988-12-26 15:52:33.1592 +2:00';
END;
```

- În acest caz diferenţa este de 2 ore.
- Două date de acest tip sunt identice dacă reprezintă aceeaşi dată în timp universal.

Tipuri Data, Timp, Interval

TIMESTAMP [(precizie)] WITH LOCAL TIME ZONE

- Similar cu tipul anterior dar la stocarea pe o coloană de tip DATE se face conversia la timpul local, deci diferenţa faţă de GMT nu este stocată.

Tipuri Data, Timp, Interval

INTERVAL YEAR [(precizie)] TO MONTH

- Se stochează intervale de ani şi luni. Precizia specifică numărul de cifre pentru an (între 0 şi 4, implicit 2). De exemplu, un interval de 2 ani şi 3 luni se poate stoca într-o variabilă de acest tip astfel:

Tipuri Data, Timp, Interval

```
DECLARE
perioada INTERVAL YEAR(1) TO MONTH;
BEGIN
perioada := INTERVAL '2-3' YEAR TO MONTH;
```

- sau:

```
perioada := '2-3';
```

- Se poate stoca numai un număr de ani sau de luni:

```
perioada := INTERVAL '2' YEAR;
perioada := INTERVAL '3' MONTH;
```

Tipuri Data, Timp, Interval

INTERVAL DAY [(precizie_zi)] TO SECOND [(precizie_sec)]

- Similar cu tipul anterior, dar pentru intervale de zile şi secunde.

Nota: Datele şi/sau intervalele se pot aduna, scădea, înmulţi rezultând date sau intervale, după caz:

Tipuri Data, Timp, Interval

```
Interval1 + Interval2 = Interval3
Interval1 - Interval2 = Interval3
Data1 + Interval = Data2
Data1 - Interval = Data2
Interval1 * numar = Interval3
Interval1 / numar = Interval3
```

Atributul %TYPE

- Putem defini tipul unei variabile în funcţie de tipul altei variabile sau a unei coloane de tabelă folosind construcţia:

`Variabila%TYPE` sau

`Tabela.Coloana%TYPE.`

- Exemplu:

```
DECLARE
   v_numar          NUMBER(5,2);
   v_altnumar            v_numar%TYPE;
   v_nume           stud.nume%TYPE;
```

- **Notă:** Dacă o variabilă este definită pe baza tipului unei coloane de tip NOT NULL ea nu moşteneşte acest atribut, putând conţine şi valoarea NULL.

Variabile externe (non PL/SQL)

- Acestea se prefixează cu două puncte (:). Exemplu:

```
:g_total := v_suma * 12;
```

- Astfel de variabile provin din mediul extern, de exemplu variabile din programele C, PHP, etc. din care se face acces la baza de date, câmpuri ale formelor sau variabile SQL*Plus.

Exemplu

```
<?php
// apelare script: prin formular sau direct (metoda GET)
  astfel:
// http://info.cs.pub.ro/~bdfr/prof/test6.php?matr=4311
// conectare si parse cerere
$c = oci_connect("scott","tiger");
$cerere = "begin
    select cods, nume into :cods, :nume from STUD
  where matr=".$_REQUEST['matr']."; ".
  "exception
          when others then
          :cods := 0; :nume := 'nu avem astfel de student';
          end;"; // atentie: nu trebuie uitat ; de dupa end!
echo "<h2>Cererea este: $cerere</h2>";
```

Exemplu

```
$stmt = oci_parse($c, $cerere);
// bind by name: se asociaza :ovariabila (din PL/SQL) cu
   $ovariabila (din PHP)
oci_bind_by_name($stmt, ":cods", $cods, 32);
oci_bind_by_name($stmt, ":nume", $nume, 32);
oci_execute($stmt,OCI_DEFAULT); // executie
if ($cods)
   echo $cods, " ",  $nume; // rezultatul va fi in $cods si
   $nume
else
   echo $nume;
?>
```

Identificatori

- Identificatorii obişnuiţi folosiţi în blocurile PL/SQL trebuie să îndeplinească următoarele condiţii:
  - Lungimea maximă este de 30 de caractere.
  - Trebuie să înceapă cu o literă.
  - Pot conţine litere, cifre şi caracterele _ $ #
- În cazul literelor nu contează dacă sunt mari sau mici. De exemplu identificatorul *NUME* este acelaşi cu *Nume* sau *nume*.
- Nu trebuie să fie identici cu numele unei coloane dintr-o tabelă a bazei de date
- Cuvintele rezervate pot fi folosite ca identificatori dar doar puse între ghilimele (exemplu "SELECT", "INSERT", "UPDATE", etc.).

Exemple

- Exemple de identificatori valizi:

```
v_nume          VARCHAR2(10);
v$nume#_        VARCHAR2(40);
v1_$            NUMBER;
```

- Exemple de identificatori invalizi:

```
1_nume          VARCHAR2(10);
v-nume&         VARCHAR2(40);
v/t=numar       NUMBER;
END             BOOLEAN;
```

Identificatori cu ghilimele

- Deşi nu sunt recomandaţi, în PL/SQL sunt permişi şi operatori care conţin orice caracter tipăribil (inclusiv spaţiu) dacă sunt puşi între ghilimele. Lungimea maximă este tot de 30 de caractere (exclusiv ghilimelele) şi în conţinutul lor nu poate apare caracterul ghilimele.
- Exemple de astfel de identificatori:

```
"1_nume"          VARCHAR2(10);
"v-nume& "        VARCHAR2(40);
"v/t=numar"       NUMBER;
"END"             BOOLEAN;
```

Identificatori cu ghilimele

- Ghilimelele sunt necesare şi atunci când în cererile SQL apar cuvinte rezervate PL/SQL dar care nu sunt rezervate în SQL.
- De exemplu, dacă o coloană a unei tabele se numeşte BEGIN (cuvânt rezervat doar în PL/SQL) atunci o cerere SQL care este parte a unui bloc PL/SQL de tipul:

```
SELECT EVENIMENT, BEGIN . . .
```

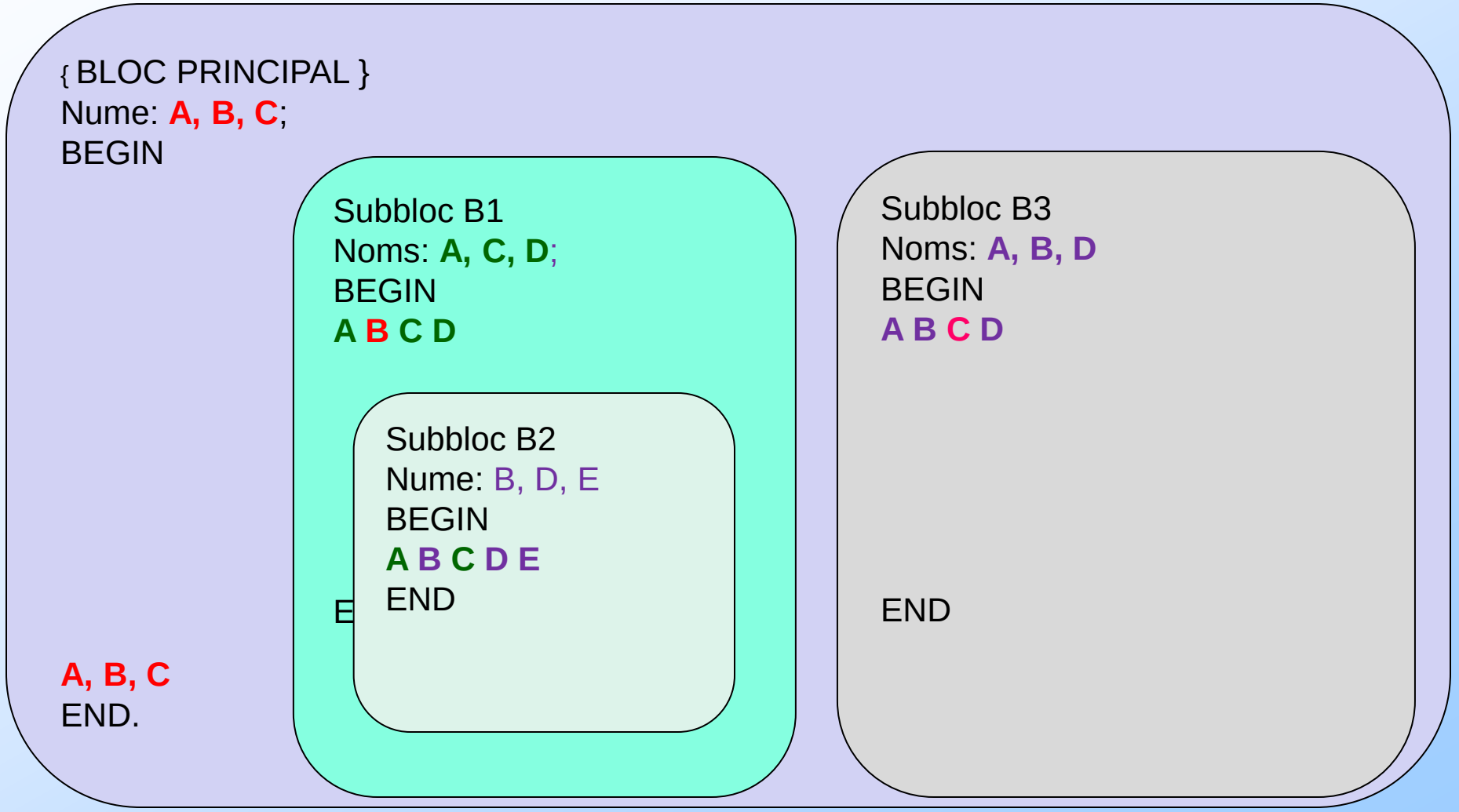
- va genera o eroare. Corect cererea trebuie scrisă astfel:

```
SELECT EVENIMENT, "BEGIN" . . .
```

Domeniul de valabilitate

- Un identificator definit într-un bloc este local acelui bloc şi global în toate subblocurile sale.
- În cazul în care un subbloc redefineşte un identificator, în subbloc poate fi folosit doar acesta din urmă.

Domeniul de valabilitate



Domeniul de valabilitate

- Exemplu:

```
DECLARE
  a NUMBER;
  b VARCHAR2(10);
BEGIN
  DECLARE
     a VARCHAR2(20);
  c NUMBER;
  BEGIN
  -- pot fi folositi a (VARCHAR2(20)), b si c
  END;
-- pot fi folositi a (NUMBER) si b
END;
```

Operatori PL/SQL

- În expresii se pot folosi operatori de comparaţie, aritmetici, logici, de concatenare precum şi cei patru operatori specifici SQL (IS NULL, LIKE, BETWEEN şi IN).
- Precedenţa operatorilor este cea din tabelul urmator:

Operatori PL/SQL

Operator	Semnificație
**	ridicare la putere
*, /	înmulțire, împărțire
+, -, ||	adunare, scădere, concatenare
=, <, >, <=, >=, <>, !=, ~=, ^=,IS NULL, LIKE, BETWEEN, IN	comparație
NOT	negație logică
AND	ȘI logic
OR	SAU logic

Operatori PL/SQL

- Pentru a preîntâmpina eventualele erori de semantică în scrierea expresiilor se recomandă folosirea parantezelor pentru a specifica explicit ordinea de efectuare a operaţiilor.
- De exemplu, următoarele două expresii logice sunt echivalente dar recomandată este a doua formă:

```
v_a > 1 and v_b < 10 or v_nr = 20
((v_a > 1) and (v_b < 10)) or (v_nr = 20)
```

Expresii

Constante (literali)

- Literalii pot fi:
- Numerici. Exemple: -12.9, 9E2 (= 9 * 10^2 = 900)
- Logici: Pot avea valoarea TRUE, FALSE şi NULL.
- Şir de caractere. Se pun între apostrofi. Exemple: 'A', 'Buna ziua', 'Don''t do that'. Între apostrofi poate fi orice caracter tipăribil. Includerea unui apostrof în şir se face prin dublarea sa, ca în ultimul exemplu
- Dată calendaristică. Se pun între apostrofi în formatul standard şi se foloseşte tipul lor generic în genul cast-ului din C (Oracle9).

Expresii

- Exemple de literali de tip dată:

```
DECLARE
d date := date '1988-05-25';
t timestamp :=
  timestamp '1988-06-20 11:00:00';
u timestamp with time zone :=
  timestamp '1988-06-20 11:00:00.25 +01:00';
i interval year to month :=
  interval '1-4' year to month;
j interval day to second :=
  interval '2 01:02:03.04' day to second;
```

- Observaţie: În Oracle v8 şirul de caractere reprezentând data nu se prefixează cu numele tipului.

Evaluare expresii logice

- În cazul unei expresii logice complexe evaluarea se opreşte în momentul în care valoarea expresiei este cunoscută.
- Acest lucru înseamnă că restul expresiei nu se mai evaluează şi deci nici nu se generează excepţii în cazul în care acestea ar fi putut să apară în partea neprocesată a expresiei.

Exemplu

```
DECLARE
   a NUMBER;
   b NUMBER;
BEGIN
      .  .  .
   if (a = 0) or ((b/a) < 1) then
      .  .  .
   END IF;
      .  .  .
END;
```

- În cazul în care variabila **a** are valoarea 0 expresia logică se evaluează la TRUE şi nu se mai testează a doua parte a ei care ar genera o excepţie de tip ZERO_DIVIDE.

Valori nule (NULL)

- Comparaţiile care implică o valoare nulă returnează NULL.
- Negarea unei valori de NULL returnează NULL.
- O condiţie care se evaluează la NULL e tratată ca FALSE.
- În cazul expresiilor CASE (prezentate mai jos) nu se poate scrie WHEN NULL ci sintaxa este WHEN expresie IS NULL.

Valori nule (NULL)

- Un şir de caractere de lungime 0 este tratat de PL/SQL ca o valoare de NULL.
- Operatorul de concatenare || ignoră valorile nule.
- Operatorul IN (listă de valori) ignoră valorile nule din listă
- Operatorul NOT IN (listă de valori) întoarce FALSE dacă lista conţine o valoare nulă.

Valori nule (NULL)

- Când o valoare de NULL este transmisă ca parametru unei funcţii SQL sau PL/SQL rezultatul este NULL în afară de cazurile următoare:
- Funcţia DECODE aflată într-o cerere SQL poate conţine argumente nule
- Funcţia NVL poate avea primul argument NULL
- Funcţia REPLACE poate avea al doilea argument NULL (întoarce primul argument indiferent de al treilea) sau al treilea argument NULL (elimină din primul argument apariţiile celui de-al doilea)

Expresii CASE

- Expresiile CASE sunt folosite pentru a returna o valoare în funcţie de una sau mai multe condiţii.
- Ele suplinesc faptul că în instrucţiunile PL/SQL nu se poate folosi funcţia DECODE (aceasta poate fi prezentă doar în cereri SQL).
- Există două forme ale acestora. Prima dintre ele este exemplificată în continuare:

Expresii CASE

```
DECLARE
   codjudet VARCHAR2(2) := 'CJ'; numejudet
  VARCHAR2(20);
BEGIN
numejudet := CASE codjudet
WHEN 'MM' THEN 'Maramures'
WHEN 'CT' THEN 'Constanta'
WHEN 'CJ' THEN 'Cluj'
ELSE 'Alt judet'    END;
END;
```

Expresii CASE

- A doua formă este o generalizare a primei forme permiţându-ne ca valoarea returnată să fie aleasă în funcţie de o condiţie generală şi nu doar de egalitate cum este în primul caz.

Expresii CASE

```
DECLARE
codjud VARCHAR2(2) := 'CJ'; numejud VARCHAR2(20);
BEGIN
numejud :=
CASE
WHEN codjud = 'MM' THEN 'Maramures'
WHEN codjud ='CT' THEN 'Constanta'
WHEN codjud ='CJ' THEN 'Cluj'
WHEN codjud LIKE 'A%' THEN 'Nume incepand cu A'
WHEN codjud IS NULL THEN 'Cod judet NULL'
ELSE 'Alt judet'
END;
END;
```

Ce urmează?

- Săptămâna viitoare vom învăţa cum se folosesc :
  - Instrucţiunile de decizie
  - Instrucţiunile de ciclare
  - Instrucţiunile de alegere (CASE)
  - Folosirea etichetelor

Sfârşit

Limbajul PL/SQL
Expresii si instructiuni.
Exceptii.

Unde ramasesem?

- Săptămâna viitoare vom învăţa cum se folosesc :
  - Instrucţiunile de decizie
  - Instrucţiunile de ciclare
  - Instrucţiunile de alegere (CASE)
  - Folosirea etichetelor

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Functii

- Expresiile SQL şi PL/SQL pot conţine funcţii din tabelul următor.
- Există însă restricţiile următoare:
  - Funcţiile SQLCODE şi SQLERRM nu pot fi folosite în cereri SQL.

Functii

Există însă restricţiile următoare:

- Funcţiile DEREF, REF, VALUE, DECODE, DUMP, GREATEST, LEAST şi VSIZE nu pot fi folosite în instrucţiunile procedurale ale PL/SQL (în documentaţia Oracle9 nu mai sunt amintite în această categorie GREATEST şi LEAST).
- Funcţiile statistice şi analitice (ex: AVG, SUM, COUNT, MIN, MAX, etc.) sunt specifice SQL şi nu pot fi folosite în instrucţiunile procedurale ale PL/SQL.

Functii

Erori	Număr	Caracter	Conversie
SQLCODE	ABS	ASCII	CHARTOROWID
SQLERRM	ACOS	CHR	CONVERT
	ASIN	CONCAT	HEXTORAW
	ATAN	INITCAP	RAWTOHEX
	ATAN2	INSTR	ROWIDTOCHAR
	BITAND	INSTRB	TO_BLOB
	CEIL	LENGTH	TO_CHAR
	COS	LENGTHB	TO_CLOB
	COSH	LOWER	TO_DATE
	EXP	LPAD	TO_MULTI_BYTE
	FLOOR	LTRIM	TO_NCLOB
	LN	NLS_INITCAP	TO_NUMBER
	LOG	NLS_LOWER	TO_SINGLE_BYTE
	MOD	NLSSORT	
	POWER	NLS_UPPER	
	ROUND	REPLACE	
	SIGN	RPAD	
	SIN	RTRIM	
	SINH	SOUNDEX	
	SQRT	SUBSTR	
	TAN	SUBSTRB	
	TANH	TRANSLATE	
	TRUNC	TRIM	
		UPPER	

Functii

Dată	Obiect	Diverse
ADD_MONTHS	DEREF	BFILENAME
CURRENT_DATE	REF	DECODE
CURRENT_TIMESTAMP	VALUE	DUMP
DBTIMEZONE	TREAT	EMPTY_BLOB
EXTRACT		EMPTY_CLOB
FROM_TZ		GREATEST
LAST_DAY		LEAST
LOCALTIMESTAMP		NLS_CHARSET_DECL_LEN
MONTHS_BETWEEN		NLS_CHARSET_ID
NEW_TIME		NLS_CHARSET_NAME
NEXT_DAY		NVL
NUMTODSINTERVAL		SYS_CONTEXT
NUMTOYMINTERVAL		SYS_GUID
ROUND		UID
SESSIONTIMEZONE		USER
SYSDATE		USERENV

Decizia

- Sintaxa este:

```
IF conditie1 THEN
   instructiuni1;
[ELSIF conditie2 THEN
   instructiuni2;]
. . . . . .
[ELSE
   instructiuni_else;]
END IF;
```

Decizia

- Se execută secvenţa de instrucţiuni corespunzătoare primei condiţii îndeplinite.
- Dacă nici o condiţie nu este adevărată şi există ELSE se execută instrucţiunile de pe această ramură.

Decizia

- Observaţie: ELSIF se scrie legat pe când END IF în două cuvinte.
- În condiţii se pot folosi toţi operatorii de comparaţie, operatorii specifici SQL (BETWEEN, IN, LIKE, IS [NOT] NULL), conectorii logici AND, OR, NOT, paranteze şi orice expresii aritmetice sau de altă natură valide.

Exemplu

```
DECLARE
  v_loc varchar2(10) := 'BUCURESTI'; v_nr   number(2);
BEGIN
  SELECT cods
  INTO v_nr
  FROM stud
  WHERE loc=v_loc;
  dbms_output.put_line('Numarul specializarii este: ' || v_nr);
  IF (v_nr=21) then
      dbms_output.put_line('Este 21');
  ELSIF v_nr < 21 then
      dbms_output.put_line('Mai mic decat 21');
  ELSE
      dbms_output.put_line('Mai mare decat 21');
  END IF;
```

Exemplu – cont.

```
EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Nu exista');
  WHEN too_many_rows THEN
    dbms_output.put_line('Sunt mai multe');
  WHEN others THEN
    dbms_output.put_line('Eroare nespecificata');
END;
```

- Rezultat:

```
Sunt mai multe
```

Ciclare: LOOP

- Sintaxa este:

```
LOOP
   Instructiuni;
         EXIT [WHEN conditie];
         -- iesire din ciclu
   Instructiuni;
END LOOP;
```

- Se execută repetat instrucţiunile dintre LOOP şi END LOOP.
- Ieşirea din ciclu se face prin EXIT (condiţionat sau nu).

Ciclare: LOOP

- Exemplu: Blocul următor conţine un ciclu LOOP cu o variabilă contor care parcurge valorile 6, 11, 16, . . . şi al cărui corp conţine un subbloc.
- Subblocul testează pentru numerele respective dacă există o specializare cu acel cod şi dacă da afişează datele despre ea în zona executabilă.
- Dacă nu există o astfel de specializare se afişează un mesaj corespunzător în zona de excepţii a subblocului.

Ciclare: LOOP

```
DECLARE
   v_contor number(2) :=6;
   v_cods number;
   v_nume varchar2(10);
   v_dom varchar2(20);
BEGIN
LOOP
   BEGIN -- subbloc
   SELECT cods, nume, domeniu
   INTO v_cods, v_nume, v_dom
   FROM spec
   WHERE cods=v_contor;
   dbms_output.put_line('Cod '|| v_contor|| ',  nume ' ||
      v_nume|| ' domeniu '||v_dom);
```

Ciclare: LOOP

```
EXCEPTION -- subbloc
    WHEN no_data_found THEN
          dbms_output.put_line('Nu exista codul '||
    v_contor);
END; -- subbloc
v_contor := v_contor + 5;
EXIT when v_contor > 40;
END LOOP;
EXCEPTION -- bloc
WHEN others THEN
  dbms_output.put_line('Exceptie');
END; -- bloc
```

Ciclare: LOOP

- Rezultat:

```
Nu exista codul 6
Cod 11  nume MATEMATICA domeniu STIINTE EXACTE
Nu exista codul 16
Cod 21  nume GEOGRAFIE domeniu UMANIST
Nu exista codul 26
Nu exista codul 31
Nu exista codul 36
```

Ciclare: FOR

- Sintaxa este:

```
FOR contor IN [REVERSE] val_min..val_max LOOP
    Instructiuni;
END LOOP;
```

Ciclare: FOR

- Se execută repetat instrucţiunile dintre FOR şi END LOOP pentru fiecare valoare a contorului.
- Acesta parcurge plaja dintre o valoare iniţială şi o valoare finală (inclusiv pe acestea). Numărul de paşi este:

  |val_max - val_min| + 1.
- Ieşirea din ciclu se face după atingerea valorii finale sau prin EXIT (pentru ieşire forţată).
- Valoarea initiala este implicit val_min.
- REVERSE specifică o parcurgere descrescătoare de la valoarea maxima către valoarea minima.

Ciclare: FOR

- Exemplu: Asemănător cu exemplul anterior dar se testează toate numerele între 20 şi 25 inclusiv.

```
DECLARE
  v_contor number(2); v_cods number;   v_nume varchar2(10);
  v_dom varchar2(20);
BEGIN
FOR v_contor IN 20..25 LOOP
  BEGIN -- subbloc
  SELECT cods, nume, domeniu
  INTO v_cods, v_nume, v_dom
  FROM spec
  WHERE cods=v_contor;
  dbms_output.put_line('Cod '|| v_contor|| ' nume '|| v_nume ||
      ' domeniu '||v_dom);
```

Ciclare: FOR

```
EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Nu exista codul '||
  v_contor);
END; -- subbloc
END LOOP;
END;
```

- Rezultat:
- **Nu exista codul 20**
- **Cod 21 nume GEOGRAFIE domeniu UMANIST**
- **Nu exista codul 22**
- **Nu exista codul 23**
- **Cod 24 nume ISTORIE domeniu UMANIST**
- **Nu exista codul 25**

Ciclare: WHILE

- Sintaxa este:

```
WHILE conditie LOOP
    Instructiuni;
END LOOP;
```

- Cât timp condiţia este adevărată se execută instrucţiunile dintre WHILE şi END LOOP.
- Se poate folosi EXIT pentru ieşirea forţată din ciclu.

Ciclare: WHILE

- Exemplu: Rescriere cu WHILE pentru exemplul de la LOOP

```
DECLARE
   v_contor number(2) :=6;
   v_cods number;
   v_nume varchar2(10);
   v_dom varchar2(20);
BEGIN
WHILE v_contor <=40 LOOP
   BEGIN -- subbloc
   SELECT cods, nume, domeniu
   INTO v_cods, v_nume, v_dom
   FROM spec
   WHERE cods=v_contor;
   dbms_output.put_line('Cod '|| v_contor|| ' nume ' ||
      v_nume ||' domeniu '||v_dom);
```

Ciclare: WHILE

```
EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Nu exista codul ‘|| v_contor);
END; -- subbloc
v_contor := v_contor + 5;
END LOOP;
EXCEPTION
WHEN others THEN
  dbms_output.put_line('Exceptie');
END;
```

Ciclare: WHILE

- Rezultatul este acelaşi cu cel obţinut anterior:

```
Nu exista codul 6
Cod 11 nume MATEMATICA domeniu STIINTE EXACTE
Nu exista codul 16
Cod 21 nume GEOGRAFIE domeniu UMANIST
Nu exista codul 26
Nu exista codul 31
Nu exista codul 36
```

Etichete

- Ciclurile se pot eticheta. Forma unei etichete este:

<<eticheta>>

- Aceasta se plasează înaintea instrucţiunii.
- Dacă instrucţiunea este un ciclu, în cazul în care avem cicluri imbricate se poate comanda ieşirea din ciclu pe mai multe niveluri specificând la EXIT eticheta ciclului părăsit.
- În acest caz la END LOOP se poate adauga opţional eticheta ciclului.

Etichete

- Exemplu:

```
<<ciclu_mare>>
LOOP
     - - -
     <<ciclu_mic>>
     LOOP
          - - -
          EXIT ciclu_mic WHEN conditie1;
          - - -
          EXIT ciclu_mare WHEN conditie2;
     END LOOP ciclu_mic;
END LOOP ciclu_mare;
```

Instructiunea CASE

- Sintaxa este:

```
CASE expresie
     WHEN 'val1' THEN
          Instructiuni1;
     WHEN 'val2' THEN
          Instructiuni2;
     - - - - - - - - -
     [ELSE
          Instructiuni_else;]
END CASE;
```

Instructiunea CASE

- Se evaluează expresia şi se execută instrucţiunile corespunzătoare valorii acesteia.
- În cazul în care expresia nu este egală cu nici una dintre valori şi există ELSE se execută instrucţiunile asociate acestei ramuri.
- Exemplu: Afişarea domeniului asociat specializării cu codul 21.

Instructiunea CASE

```
DECLARE
  v_dom varchar2(20);
BEGIN
  SELECT domeniu INTO v_dom FROM spec WHERE cods=21;
  CASE v_dom
    WHEN 'UMANIST' THEN
      dbms_output.put_line('DOMENIU UMANIST');
    WHEN 'STIINTE EXACTE' THEN
      dbms_output.put_line('DOMENIU STIINTE EXACTE');
    ELSE
      dbms_output.put_line('ALT DOMENIU');
  END CASE;
END;
Rezultat:
DOMENIU UMANIST
```

Instructiunea CASE

- În cazul absenţei lui ELSE se adaugă implicit ridicarea unei excepţii predefinite:

```
ELSE RAISE CASE_NOT_FOUND;
```

- Exemplu: domeniul pentru specializarea cu cod 21 (UMANIST) nu este printre etichetele CASE şi nu există ELSE. In consecinta va semnala excepţia:
- ORA-06592: CASE not found while executing CASE statement.

- Notă: instrucţiunea CASE apare în Oracle versiunea 9.

Instructiunea CASE

```
DECLARE
  v_dom varchar2(20);
BEGIN
  SELECT domeniu
  INTO v_dom
  FROM spec
  WHERE cods=21;
  CASE v_dom
    WHEN 'MATEMATICA' THEN
      dbms_output.put_line('DOMENIU UMANIST');
    WHEN 'STIINTE EXACTE' THEN
      dbms_output.put_line('EXACTE');
  END CASE;
END;
```

Exceptii

- În cazul sistemelor de gestiune a bazelor de date erorile (dar nu numai) sunt numite şi ***excepţii***. Un bloc PL/SQL poate conţine între EXCEPTION şi END instrucţiuni care să trateze :
  - Erorile returnate de serverul Oracle sau uneltele Oracle
  - Situaţii definite de utilizator (excepţii definite de utilizator).

Exceptii

- Excepţiile pot avea asociat un nume (identificator). Unele dintre aceste nume sunt predefinite (pentru o parte dintre erorile returnate de server sau unelte).
- Apariţia unei excepţii este numită şi ***ridicarea unei excepţii*** (exception raising). În acest caz execuţia normală a blocului este întreruptă şi se face ***tratarea excepţiei***: se sare în zona EXCEPTION pentru execuţia instrucţiunilor asociate acesteia.

Exceptii

- Caracteristicile principale ale tratării excepţiilor Oracle sunt următoarele:

1. Apariţia unei excepţii, tratată sau nu, termină execuţia blocului – nu se revine la continuarea executiei!
- Toate instrucţiunile PL/SQL sau cererile SQL care apar după cererea sau instrucţiunea care a ridicat excepţia nu mai sunt executate.

Exceptii

- Caracteristicile principale ale tratării excepţiilor Oracle sunt următoarele:

2. În cazul în care o excepţie are asociat un tratament, la ridicarea ei se trece la execuţia instrucţiunilor de tratare ***după care se iese din blocul curent fără eroare.***

Exceptii

- Caracteristicile principale ale tratării excepţiilor Oracle sunt următoarele:

3. În cazul în care apare o excepţie care nu este tratată explicit în porţiunea dintre EXCEPTION şi END, ***blocul se termină cu eroare***, excepţia putând fi tratată de blocul înconjurător (dacă există). Apare deci o ***propagare a exceptiei***.

Exceptii

- Caracteristicile principale ale tratării excepţiilor Oracle sunt următoarele:

4. În cazul excepţiilor definite de utilizator acestea trebuie ***ridicate explicit*** în zona dintre BEGIN şi EXCEPTION prin instrucţiunea PL/SQL RAISE.

Exemplu

- Un exemplu de situaţie în care serverul returnează o eroare este următorul:

```
DECLARE
     v_nume   varchar2(10);
BEGIN
     SELECT nume INTO v_nume FROM stud
     WHERE cods>20;
END;
```

Exemplu

- Cererile SELECT .. INTO trebuie sa returneze exact o linie!
- În cazul în care cererea SQL returnează mai mult de o singură înregistrare, valoarile rezultate nu pot fi stocate într-o singură variabilă scalară.
- O altă eroare pentru blocul de mai sus este şi nereturnarea nici unei valori de către cererea SQL (în cazul în care nici o specializare nu are codul mai mare ca 20).

Tratarea exceptiilor

- Sintaxa tratării excepţiilor este următoarea:

```
DECLARE
. . . .
BEGIN
. . . .
EXCEPTION
WHEN exceptie_11 OR exceptie_12 . . .THEN
      instructiuni_1
[WHEN exceptie_21 OR exceptie_22 . . . THEN
      instructiuni_2]
.   .   .   .   .   .
[WHEN OTHERS THEN
      Instructiuni_others]
END;
```

Tratarea exceptiilor

- **exceptie_xy** este identificatorul unei excepţii (predefinit sau definit de utilizator). OR se foloseşte când mai multe excepţii au aceleaşi instrucţiuni de tratare.
- **instructiuni_x** reprezintă secvenţa de instrucţiuni PL/SQL sau cereri SQL care se execută la apariţia excepţiilor exceptie_xy.
- **instructiuni_others** reprezintă secvenţa de instrucţtiuni PL/SQL sau cereri SQL care se execută la apariţia unei excepţii netratate de clauzele WHEN anterioare.

Tratarea exceptiilor

- De reţinut:
  - WHEN OTHERS, dacă există, este întotdeauna ultima.
- După tratatea excepţiei apărute se iese din blocul curent.
  - În consecinţă este inutilă prezenţa aceleiaşi excepţii pe mai mult de un WHEN.

Tipuri de exceptii

- Excepţiile sunt de trei feluri:

1. Excepţii Oracle predefinite.
2. Excepţii Oracle care nu sunt predefinite
3. Excepţii definite de utilizator

Excepţii Oracle predefinite

- Excepţiile predefinite sunt erori Oracle care au deja un identificator asociat.
- Ele sunt ridicate automat la apariţie de către serverul Oracle sau uneltele Oracle.

- Unele dintre excepţiile predefinite ridicate de serverul Oracle (* = incepand cu Oracle9)

Excepţii Oracle predefinite

Numele excepţiei	Când este ridicată de serverul Oracle
ACCESS_INTO_NULL	Asignarea unei valori unui atribut al unui obiect neiniţializat
CASE_NOT_FOUND (*)	Instrucţiune CASE fără ELSE pentru care nici una dintre etichete nu este egală cu valoarea căutată
COLLECTION_IS_NULL	Aplicarea unei alte metode decat EXISTS unui TABLE sau VARRAY neiniţializat
CURSOR_ALLREADY_OPEN	Încercare de deschidere a unui cursor deja deschis
DUP_VAL_ON_INDEX	Inserarea unei valori duplicat pe o coloană unde acest lucru nu este permis (PRIMARY KEY, UNIQUE, etc)
INVALID_CURSOR	Operaţie cursor invalidă
INVALID_NUMBER	Eroare conversie de la şir de caractere la număr

Excepţii Oracle predefinite

Numele excepţiei	Când este ridicată de serverul Oracle
LOGIN_DENIED	Autentificare Oracle cu un username/parolă invalide
NO_DATA_FOUND	Cererea SELECT care ar fi trebuit să întoarca o singură linie nu întoarce nici o linie
NOT_LOGGED_ON	Programul PL/SQL execută un acces la server fără să fie realizată în prealabil conexiunea cu acesta
PROGRAM_ERROR	Eroare internă PL/SQL
ROWTYPE_MISMATCH	Diferenţa de tip între două cursoare (unul gazdă, altul PL/SQL) implicate într-o asignare
SELF_IS_NULL (*)	Programul apelează o metodă MEMBER pe o instantă nulă. În acest caz, parametrul implicit SELF - primul transmis unei metode MEMBER - e nul
STORAGE_ERROR	Depăşire memorie pentru PL/SQL sau memoria este coruptă

Excepţii Oracle predefinite

Numele excepţiei	Când este ridicată de serverul Oracle
SUBSCRIPT_BEYOND_COUNT	Referinţa la o tabelă imbricată sau un obiect de tip VARRAY printr-un index mai mare decât numărul de elemente al colecţiei
SUBSCRIPT_BEYOND_LIMIT	Referinţa la o tabelă imbricată sau un obiect de tip VARRAY printr-un index ilegal (de exemplu -1)
SYS_INVALID_ROWID (*)	Conversia unui şir de caractere la tipul UROWID a întors eroare deoarece şirul nu este un UROWID valid.
TIMEOUT_ON_RESOURCE	Time-out la încercarea de accesare a unei resurse de către Oracle
TOO_MANY_ROWS	Cererea SELECT care ar fi trebuit să întoarcă o singură linie întoarce mai multe linii
VALUE_ERROR	Erori aritmetice, de conversie, de trunchiere sau de violare a unei constrângeri privind o valoare
ZERO_DIVIDE	Împărţire la 0

Exemplu

- Blocul următor va tipări mesajul 'Mai multe linii'.

```
DECLARE
v_nume varchar2(10);
BEGIN
SELECT nume INTO v_nume FROM stud WHERE cods>20;
EXCEPTION
       WHEN NO_DATA_FOUND THEN
              DBMS_OUTPUT.PUT_LINE('Nu exista date');
       WHEN TOO_MANY_ROWS THEN
              DBMS_OUTPUT.PUT_LINE('Mai multe linii');
       WHEN OTHERS THEN
              DBMS_OUTPUT.PUT_LINE('Alta exceptie');
END;
```

De retinut!

Excepţiile predefinite:

1. NU se declară în zona DECLARE
2. NU trebuiesc ridicate explicit prin RAISE
3. Se tratează în zona EXCEPTION

Excepţii Oracle non-predefinite

- Reprezintă alte erori returnate de Oracle, în afara celor predefinite. Aceste erori se pot trata în două moduri:

1. Cu WHEN OTHERS, prin prelevarea codului numeric de eroare si apoi CASE ...
2. Prin nume definit de noi:
   - declararea excepţiei (ca nume) şi asocierea ei cu un cod de eroare Oracle în zona DECLARE şi
   - tratarea ei în zona EXCEPTION.

Excepţii Oracle non-predefinite

- Declarare exceptie:

```
DECLARE
nume_exceptie      EXCEPTION;
PRAGMA EXCEPTION_INIT(nume_exceptie, cod_eroare_Oracle);
BEGIN
.   .   .   .
EXCEPTION
      WHEN nume_exceptie THEN
            instructiuni;
           .   .   .   .
END;
```

- Exemplu: Tratarea erorii cu cod -979 (în SELECT există coloane care nu apar în GROUP BY):

Exemplu

```
DECLARE
      v_nume VARCHAR2(10);
      v_numar NUMBER;
      eroare_grupare  EXCEPTION;
      PRAGMA EXCEPTION_INIT(eroare_grupare, -979);
BEGIN
   SELECT NUME, COUNT(*) INTO v_nume, v_numar
   FROM STUD GROUP BY CODS;
EXCEPTION
         WHEN eroare_grupare THEN
            DBMS_OUTPUT.PUT_LINE('Cerere gresita');
         WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie');
END;
```

De retinut!

Excepţii Oracle non-predefinite

1. Se declară în zona DECLARE şi se asociază un cod de eroare
2. NU trebuiesc ridicate explicit prin RAISE
3. Se tratează în zona EXCEPTION

Excepţii definite de utilizator

- Un program PL/SQL poate conţine excepţii definite de utilizator. În cazul când programul detectează o situaţie anormală el poate trece în zona de excepţii prin instrucţiunea PL/SQL

```
RAISE nume_exceptie
```

- Astfel de excepţii trebuiesc:
  - Declarate explicit
  - Ridicate explicit

Exemplu

```
DECLARE
      prea_putini EXCEPTION;
      v_nrstud    NUMBER;
BEGIN
      SELECT count(*) INTO v_nrstud FROM stud WHERE cods=21;
      IF (v_nrstud < 5) THEN
            RAISE prea_putini;
      END IF;
EXCEPTION
      WHEN prea_putini THEN
            DBMS_OUTPUT.PUT_LINE('Sunt sub 5 studenti');
      WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Alta exceptie');
END;
```

De retinut!

Excepţii definite de utilizator

1. Se declară în zona DECLARE (ca nume)
2. Se ridică explicit cu RAISE
3. Se tratează în zona EXCEPTION

Identificarea unei excepţii

- În cazul în care dorim ca în zona EXCEPTION .. WHEN OTHERS să identificăm ce eroare a apărut putem folosi două funcţii care întorc codul numeric respectiv mesajul text al erorii respective.
- În funcţie de valoarea lor putem să tratăm diferenţiat diferite erori.

Identificarea unei excepţii

- Cele două funcţii sunt:

1. SQLCODE - întoarce codul numeric al excepţiei (erorii returnate)
2. SQLERRM - întoarce mesajul text asociat cu acea excepţie

Identificarea unei excepţii

- Codul de eroare returnat poate fi:

Valoare SQLCODE	Descriere
0	Nu a apărut nici o excepţie
1	A apărut o excepţie definită de utilizator
100	A apărut excepţia NO_DATA_FOUND
Număr negativ	Cod eroare returnat de serverul Oracle

Exemplu

```
DECLARE
        v_cod NUMBER;
        v_text        VARCHAR2(255);
BEGIN
.   .   .   .   .
EXCEPTION
           WHEN OTHERS THEN
                ROLLBACK;
                v_cod := SQLCODE;
                v_text := SQLERRM;
                INSERT INTO ERORI VALUES(v_cod, v_text);
END;
```

Sfârşit

Administrarea bazelor de date

PL/SQL - CURS 3: Exceptii (2); Tipurile RECORD si TABLE ... INDEX BY

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Domeniul de valabilitate excepţii

- O excepţie definită într-un bloc este locală blocului respectiv şi globală pentru toate subblocurile acestuia (dacă nu este redefinită), dar nu este cunoscută în afara blocului în care e declarată.
- În cazul în care un bloc tratează excepţiile apărute, el se termină fără eroare şi procesarea continuă cu instrucţiunile care urmează după END-ul blocului respectiv.

Domeniul de valabilitate excepţii

- În cazul în care însă un bloc nu tratează o anumită excepţie în propria sa zonă EXCEPTION aceasta se propagă la blocurile înconjurătoare până găseşte tratarea adecvată.
- Exemplu: Două excepţii cu acelaşi nume sunt definite în bloc şi în subbloc. Ele sunt considerate diferite: prima aparţine blocului fără subbloc iar a doua doar subblocului.
- În cazul urmator excepţia ridicată în subbloc nu este tratată de blocul înconjurător.

Exemplu

```
DECLARE
exceptia1 EXCEPTION;
BEGIN
   DECLARE -- incepe un subbloc
   exceptia1 EXCEPTION; -- redeclarare in subbloc
   BEGIN
       ...
       IF ... THEN
             RAISE exceptia1; -- nu e tratata de bloc
             -- fiind redefinita in subbloc
       END IF;
   END; -- sfarsit subbloc
EXCEPTION
WHEN exceptia1 THEN -- nu trateaza exceptia din subbloc
...
END;
```

Domeniul de valabilitate excepţii

- Pentru a trata excepţia apărută în subbloc se poate proceda în mai multe moduri:

1. Se elimină declaraţia excepţiei din subbloc în care caz ea va fi tratată de clauza WHEN existentă în bloc
2. Se adaugă o zonă EXCEPTIONS în subbloc în care să se trateze această excepţie, tratare care poate fi diferită de cea existentă în bloc.
3. Se adaugă WHEN OTHERS la excepţiile blocului în care caz toate excepţiile netratate în subbloc şi bloc vor putea fi tratate aici.

RAISE_APPLICATION_ERROR

- Excepţiile de acest tip sunt similare erorilor Oracle non-predefinite dar trebuiesc ridicate explicit în zona executabilă sau în zona de excepţii folosind (sunt erori definite de user):

```
RAISE_APPLICATION_ERROR(cod_eroare, text_eroare)
```

unde:

- **cod_eroare** este un număr ales de utilizator între -20000 şi -20999
- **text_eroare** este un text ales de utilizator

- Şi la aceste excepţii se poate folosi PRAGMA EXCEPTION_INIT pentru a le asigna un nume!

Exemplu

```
DECLARE
   v_suma   NUMBER;
   suma_nula      EXCEPTION;
   PRAGMA EXCEPTION_INIT(suma_nula, -20021);
BEGIN
   SELECT suma INTO v_suma
   FROM bursa
   WHERE pmin < 100;
   IF v_suma IS NULL THEN
       RAISE_APPLICATION_ERROR(-20021, 'Suma este nula');
   END IF;
EXCEPTION
   WHEN suma_nula THEN
         dbms_output.put_line(SQLERRM);
END;
```

RAISE_APPLICATION_ERROR

- În cazul în care nu are asociat un nume, excepţia se poate trata în zona WHEN OTHERS folosind SQLCODE:

Exemplu

```
DECLARE
   v_suma   NUMBER;
   v_coderoare NUMBER;
BEGIN
   SELECT suma INTO v_suma
   FROM bursa WHERE pmin < 100;
   IF v_suma IS NULL THEN
      RAISE_APPLICATION_ERROR(-20021, 'Suma este nula');
   END IF;
EXCEPTION
   WHEN OTHERS THEN
      v_coderoare := SQLCODE;
      IF (v_coderoare = -20021) THEN
                dbms_output.put_line(SQLERRM);
      END IF;
END;
```

Retratarea excepţiilor

- În cazul în care se doreşte tratarea unei excepţii apărute într-un subbloc atât în acesta cât şi în blocul înconjurător, se procedează în modul următor:
  - Se prevede tratarea excepţiei respective în zona EXCEPTIONS a subblocului.
  - La sfârşitul secvenţei de instrucţiuni care tratează excepţia se plasează instrucţiunea RAISE fără parametri. Ea propaga excepţia curentă spre blocul înconjurător.
  - Se prevede tratarea excepţiei respective în zona EXCEPTIONS a blocului.

Exemplu

```
DECLARE
   exceptia1 EXCEPTION;
BEGIN
   DECLARE -- incepe un subbloc
   BEGIN
   ...
   IF ... THEN
        RAISE exceptia1; -- nu e tratata de bloc
                                -- fiind redefinita in subbloc
   END IF;
   EXCEPTION -- tratare exceptii subbloc
        WHEN exceptia1 THEN
        -- instructiuni tratare in subbloc
        .  .  .  .
        RAISE;
   END; -- sfarsit subbloc
EXCEPTION
WHEN exceptia1 THEN -- retratare exceptia1 in bloc
...
END;
```

Notă: RAISE fără parametri poate sa apara doar în zona de tratare a excepţiei!

Excepţii în declaraţii

- Excepţiile care apar în zona DECLARE determină terminarea execuţiei blocului curent şi nu pot fi tratate în zona EXCEPTIONS a acestuia.
- În schimb ele pot fi tratate în blocul înconjurător (dacă există)!
- Un exemplu de astfel de excepţie este iniţializarea necorespunzătoare a unei variabile (ca mai jos unde într-o constantă de 3 cifre se încearcă stocarea unui număr de 4 cifre)

```
sal_minim CONSTANT NUMBER(3) := 1200;
```

Excepţii în tratarea excepţiilor

- În zona de tratare a excepţiilor se poate ridica aceeaşi excepţie (cum a fost descris mai sus) sau o altă excepţie.
- În toate cazurile excepţia respectivă poate fi tratată doar în blocul înconjurător şi nu în blocul curent

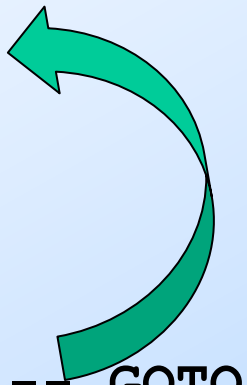
GOTO

- Instrucţiunea GOTO nu poate fi folosită pentru a se trece din zona de tratare a excepţiilor în zona executabilă a blocului respectiv şi nici invers:

```
GOTO <<eticheta>>
```

- Exemplu de GOTO ilegal:

```
BEGIN
<<o_eticheta>>
SELECT .   .   .;
EXCEPTION
WHEN . . . THEN
GOTO <<o_eticheta>>; -- GOTO ilegal
END;
```



- Ea poate fi folosită însă pentru salt din zona de tratare a excepţiilor în zona executabilă a unui bloc înconjurător!

Toate instrucţiunilor blocului

- În cazul în care apare o excepţie, restul instrucţiunilor blocului nu se mai execută.
- Dacă se doreşte execuţia acestora şi în cazul apariţiei unei excepţii, instrucţiunile care cauzează astfel de eveniment trebuiesc transformate în subblocuri.
- Să considerăm următorul bloc:

```
DECLARE
.  .  .
BEGIN
  INSERT . . . ;
  UPDATE . . . ;
  SELECT . . . ;
EXCEPTION
. . .
END;
```

Toate instrucţiunilor blocului

- În cazul apariţiei unei excepţii la INSERT următoarele două cereri SQL nu se mai execută.
- La fel, în cazul apariţiei unei excepţii la UPDATE nu se mai execută SELECT-ul.
- Soluţia este includerea fiecarei astfel de instructiuni intr-un subbloc care isi trateaza exceptiile.
- Daca apare o eroare, ea este tratata si subblocul se termina cu succes. In acest fel se executa toate instructiunile blocului.

Exemplu

```
DECLARE
.  .  .
BEGIN
   BEGIN -- subbloc pentru INSERT
        INSERT . . . ;
   EXCEPTION -- exceptii subbloc INSERT
       WHEN OTHERS THEN
            . . .
   END; -- end subbloc INSERT
   BEGIN -- subbloc pentru UPDATE
        UPDATE . . . ;
   EXCEPTION -- exceptii subbloc UPDATE
       WHEN OTHERS THEN
            . . .
   END; -- end subbloc UPDATE
   SELECT . . . ;
EXCEPTION -- exceptii pentru bloc
. . .
END; -- end bloc
```

Azi: TIPURI COMPUSE:

- RECORD
- TABLE .. INDEX BY

Declarare

- Pentru descrierea tipurilor compuse se poate folosi declaraţia de tip. Aceasta are următoarea formă:

```
TYPE denumire_tip descriere_tip;
```

- Cu ajutorul ei vom putea defini şi tipurile compuse care se pot folosi în PL/SQL:

1. Înregistrare (RECORD)
2. Tablou asociativ (TABLE .. INDEX BY)
3. Tabelă (Nested TABLE)
4. Tablou de dimensiune variabilă (VARRAY)

Declarare

- Ultime trei tipuri de date compuse de mai sus sunt denumite generic ***colecţii***.
- Acest curs prezintă primele doua tipuri compuse: înregistrările şi tablourile asociative.

Înregistrări (RECORD)

- Înregistrările din PL/SQL sunt similare înregistrărilor din limbajul Pascal sau structurilor din C.
- Sub un acelaşi nume sunt înmagazinate mai multe valori de tipuri diferite care se numesc ***câmpuri***.

Înregistrări (RECORD)

- Definirea unei înregistrări se poate face în două moduri:
  - Folosind descrierea fiecărei componente a înregistrării. În acest caz o variabilă de acest tip poate să nu fie asociată cu o tabelă sau cu rezultatul unei cereri SQL.
  - Folosind %ROWTYPE se poate defini o înregistrare compatibilă cu o linie a unei tabele sau cu o linie a rezultatului unei cereri SQL.

Descrierea componentelor

- În acest caz sintaxa definirii unui tip este următoarea:

```
TYPE denumire_tip IS RECORD
(nume_camp tip_camp [NOT NULL]
                    [:= | DEFAULT
  expresie]
[, nume_camp ...]);
```

Descrierea componentelor

- Câmpurilor li se poate asocia o valoare iniţială (prin **:=** sau **DEFAULT**) şi se poate specifica faptul că nu sunt permise valori nule (**NOT NULL**).
- Tipul unui câmp poate fi oricare dintre tipurile scalare sau compuse din PL/SQL în afară de REF CURSOR.
- Referirea unui câmp al unei variabile de tip înregistrare se face prin construcţia:

```
nume_inregistrare.nume_camp
```

Exemplu

```
DECLARE
  v_punctaj              NUMBER(4);
  TYPE t_student IS RECORD
      (matr       NUMBER(4) NOT NULL := 1234,
      nume        stud.nume%type,
      punctaj     v_punctaj%type,
      data        DATE DEFAULT SYSDATE);
  v_student t_student;

BEGIN
  v_student.matr := v_student.matr + 1;
  v_student.nume := 'MARCEL';
  v_student.punctaj := 1400;
  .   .   .
END;
```

Exemplu

- Mai sus a fost definit un tip de înregistrare numit ***t_student*** şi o variabilă de acest tip (***v_student***).
- Se observă că se poate defini tipul unui câmp şi în funcţie de tipul unei variabile sau al unei coloane a unei tabele.
- În exemplul de mai sus toate câmpurile înregistrării sunt scalare dar în cazul general ele pot fi şi compuse (de exemplu înregistrare, definită inclusiv cu %ROWTYPE, descris în continuare).

Definite cu %ROWTYPE

- În acest caz sintaxa definiţiei unei variabile înregistrare este:

```
nume_variabila   nume_tabela%ROWTYPE;
```

- Înregistrarea va avea atâtea câmpuri câte coloane are tabela şi acestea se vor numi la fel cu aceste coloane.
- Astfel de înregistrări se pot utiliza pe clauza INTO a unei cereri SELECT * care întoarce o singură linie.

Exemplu

```
DECLARE
  v_student stud%ROWTYPE;
BEGIN
  SELECT *   INTO v_student
  FROM stud  WHERE matr=1456;
  v_student.nume := 'MARCEL';
  v_student.punctaj := 1400;
  .   .   .
END;
```

Tablouri asociative (TABLE . . . INDEX BY)

- Tablourile asociative stochează perechi (cheie, valoare) unde valorile sunt de acelaşi tip.
- Nu au dimensiune fixată la declarare şi fiecare componentă se creează la prima asignare.
- Caracteristici:

1. Când sunt numerice, cheile nu e obligatoriu să fie consecutive şi nici să înceapă de la 1.
- În Oracle8 cheia putea fi doar BINARY_INTEGER, dar începând cu Oracle9 ea poate fi şi VARCHAR2, un subtip al VARCHAR2 sau un tip convertibil la VARCHAR2.

Tablouri asociative (TABLE . . . INDEX BY)

- Caracteristici:

2. În Oracle8 valorile puteau fi scalare sau de tip ROWTYPE.
- Începând cu Oracle9 poate fi orice tip definit anterior.
3. Elementele unei astfel de colecţii nu pot fi iniţializate la declarare.

Tablouri asociative (TABLE . . . INDEX BY)

- Caracteristici:

4. Tablourile asociative sunt concepute pentru a stoca date temporare ale programelor PL/SQL.
- De aceea ele nu pot fi utilizate în comenzi SQL ca SELECT sau INSERT pentru a fi regăsite/stocate în baza de date.
5. Din punct de vedere al echivalenţei cu structuri de date folosite în alte limbaje de programare, ele sunt similare tabelelor de dispersie (hash tables).

Tipuri admise

- Tipuri admise pentru cheie:
  - BINARY_INTEGER sau PLS_INTEGER.
  - VARCHAR2(n) sau subtipurile sale: VARCHAR, STRING, LONG.
  - Tipuri convertibile la VARCHAR2 cum sunt DATE, TIMESTAMP.
- Tipuri care nu sunt admise pentru cheie:
  - RAW, LONG RAW, ROWID
  - CHAR, CHARACTER.

Declarare

- Sintaxa declarării unui tablou asociativ:

```
TYPE nume_tip IS TABLE OF
     tip_element [NOT NULL]
INDEX BY [BINARY_INTEGER |
          PLS_INTEGER |
          VARCHAR2(dim_max)];
```

Metode

- Colecţiile au asociate o serie de ***metode***.
- Acestea sunt proceduri şi funcţii puse la dispoziţie de sistem pentru orice variabilă de acest tip iar apelul lor se face prin construcţia:

```
variabila.metoda[(parametri)]
```

- In cazul tablourilor asociative se pot folosi următoarele metode:

Metode

Metoda	Efect
`EXISTS(n)`	TRUE dacă elementul cu cheia ***n*** există (este setat)
`COUNT`	Numărul de elemente din tablou.
`FIRST, LAST`	Valoarea **cheii** pentru primul, respectiv ultimul element din tablou.
`PRIOR(n),` **`NEXT(n)`**	Valoarea **cheii** care precede respectiv succede în tablou elementul cu cheia ***n***. Dacă nu există un astfel de element întoarce NULL.
`DELETE[(n[,k)]]`	DELETE: şterge toate **elementele** tabloului. DELETE(n) şterge elementul cu cheia ***n*** DELETE(n, k) şterge toate elementele cu cheia în plaja ***n . . k***

Exemplu

```
DECLARE
TYPE t_varsta IS TABLE OF NUMBER INDEX BY VARCHAR2(30);
TYPE t_nrstud IS TABLE OF NUMBER INDEX BY BINARY_INTEGER;
TYPE t_stud IS TABLE OF stud%ROWTYPE
  INDEX BY BINARY_INTEGER;
v_varsta   t_varsta;
v_nrstud   t_nrstud;
v_stud     t_stud;
BEGIN
v_varsta('Ionescu') := 23;
v_varsta('Badea') := 25;
v_varsta('Avramescu') := 27;
```

Exemplu

```
dbms_output.put_line(v_varsta.FIRST);
-- Avramescu
dbms_output.put_line(v_varsta.LAST);
-- Ionescu
v_nrstud(1211) := 48; -- Indicii nu incep de la 1
v_nrstud(1221) := 29;
v_nrstud(1231) := 20;
v_nrstud(1241) := 21;
v_nrstud(1251) := 17; -- sunt 5 componente
v_stud(4875).nume := 'MARCEL';
v_stud(3456).an   := 1;
v_stud(3456).punctaj  := 1400;  -- 2 componente
END;
```

Exemplul 2

```
DECLARE
TYPE t_nrstud IS TABLE OF NUMBER INDEX BY
  BINARY_INTEGER;
TYPE t_stud IS TABLE OF stud%ROWTYPE INDEX BY
  BINARY_INTEGER;
v_nrstud          t_nrstud;
v_stud             t_stud;
BEGIN
  v_nrstud(1211) := 48;  -- Cheile nu incep de la 1
  v_nrstud(1221) := 29;
  v_nrstud(1231) := 20;
  v_nrstud(1241) := 21;
  v_nrstud(1251) := 17;              -- 5 componente
```

Exemplul 2

```
dbms_output.put_line('FIRST: ' || v_nrstud.FIRST);
  dbms_output.put_line('LAST: ' ||v_nrstud.LAST);
  dbms_output.put_line('COUNT: ' ||v_nrstud.COUNT);
if v_nrstud.EXISTS(1200) then
  dbms_output.put_line('Exista cheia 1200');
else
  dbms_output.put_line('Nu exista cheia 1200');
end if;
if v_nrstud.EXISTS(1231) then
  dbms_output.put_line('Exista cheia 1231');
else
  dbms_output.put_line('Nu exista cheia 1231');
end if;
```

Exemplul 2

```
dbms_output.put_line('PRIOR(1241): '
  ||v_nrstud.PRIOR(1241));
dbms_output.put_line('PRIOR(1211): '
  ||v_nrstud.PRIOR(1211));
    -- valoarea lui PRIOR este NULL
dbms_output.put_line('NEXT(1241): '
  ||v_nrstud.NEXT(1241));
dbms_output.put_line('NEXT(1251): '
  ||v_nrstud.NEXT(1251));
    -- valoarea lui NEXT este NULL
v_nrstud.DELETE(1);     -- cheia nu exista
dbms_output.put_line('Stergere element cu cheia
  1. Raman:' ||v_nrstud.COUNT|| ' elemente');
```

Exemplul 2

```
v_nrstud.DELETE(1241);  -- cheie existenta
dbms_output.put_line('Au ramas: '||v_nrstud.COUNT||'
  elemente');
v_nrstud.DELETE(1220, 1300);  -- sterge chei in
                              -- plaja data
dbms_output.put_line('Au ramas: '||v_nrstud.COUNT||'
  elemente');
v_stud(4875).nume := 'MARCEL';
v_stud(3456).an   := 2;
v_stud(3456).punctaj  := 2400;
                -- sunt 2 chei distincte
dbms_output.put_line('COUNT: '||v_stud.COUNT);
                -- afiseaza 2
END;
```

Exemplul 2

- Rezultatul execuţiei este:

```
FIRST: 1211
LAST: 1251
COUNT: 5
Nu exista cheia 1200
Exista cheia 1231
PRIOR(1241): 1231
PRIOR(1211):
NEXT(1241): 1251
NEXT(1251):
Stergere element cu cheia 1. Raman:5 elemente
Au ramas: 4 elemente
Au ramas: 1 elemente
COUNT: 2
```

Sfârşitul cursului 3

Administrarea bazelor de date

CURS PL/SQL 4: Cursori

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Ce sunt cursorii

- Pentru execuţia cererilor SQL prezente în blocurile PL/SQL şi pentru stocarea rezultatelor acestora (cum e cazul cererilor SELECT) Oracle foloseşte un spaţiu de lucru accesibil utilizatorului prin intermediul unui obiect numit cursor.
- Acesta poate fi de exemplu folosit pentru a parcurge linie cu linie rezultatul şi pentru a afla câte linii are acesta.

Tipuri de cursori

- Cursorii sunt de două tipuri:

1. **Cursori impliciţi**. Ei sunt definiţi automat de Oracle pentru orice cerere de tip DML, inclusiv pentru cererile SELECT .. INTO care returnează o singură linie prezentate în capitolele anterioare.
   - Aceşti cursori nu au un nume declarat de utilizator iar cererea SQL asociată lor este ultima cerere SQL executată.
2. **Cursori expliciţi**. Aceştia sunt definiţi de utilizator care precizează numele cursorului şi cererea SQL asociată.

Utilizare

- Pentru a folosi cursori expliciţi paşii sunt următorii:

1. În zona de declaraţii se defineşte cursorul (nume, cerere SQL asociată).
2. În zona executabilă (dupa BEGIN, deci inclusiv în zona de tratare a excepţiilor) se pot folosi instrucţiunile de lucru cu un cursor: OPEN, FETCH şi CLOSE.
3. Pentru orice tip de cursor se pot folosi metodele asociate acestora: %ISOPEN, %NOTFOUND, %FOUND şi %ROWCOUNT

Observaţii

- Instrucţiunile OPEN, FETCH şi CLOSE sunt specifice cursorilor expliciţi şi nu pot fi folosite pentru cei impliciţi (pentru aceştia operaţiile respective sunt executate automat de Oracle).
- Pentru cursorii impliciţi valorile atributelor acestora (%NOTFOUND, %FOUND şi %ROWCOUNT) se pot afla folosind un nume de cursor generic - *SQL* - care desemnează cel mai recent cursor implicit deschis.

Cursori impliciti

- Ultima cerere executată a unui bloc PL/SQL are asociat un cursor cu numele generic SQL.
- Trebuie remarcat că spre deosebire de cererile SELECT . . INTO, celelalte cereri DML nu generează excepţii în cazul în care nici o linie nu este afectată (de exemplu în cazul DELETE sau UPDATE).

Cursori impliciti

- Atributele cursorului SQL vor avea următoarele valori:
  - SQL%ROWCOUNT: numărul de linii afectate de cerere
  - SQL%FOUND: TRUE dacă există linii afectate de cerere şi FALSE altfel
  - SQL%NOTFOUND: TRUE dacă nu există linii afectate de cerere şi FALSE altfel
  - SQL%ISOPEN: nu se aplică cursorilor impliciţi.

Exemplu

```
BEGIN
UPDATE STUD
SET PUNCTAJ = PUNCTAJ + 100
WHERE CODS = 21;
dbms_output.put_line('Au fost modificate' ||
  SQL%ROWCOUNT || ' linii');
IF sql%found THEN
   dbms_output.put_line('Found TRUE');
ELSE
   dbms_output.put_line('Found FALSE');
END IF;
```

Exemplu

```
IF sql%notfound THEN
  dbms_output.put_line('NotFound TRUE');
ELSE
  dbms_output.put_line('NotFound FALSE');
END IF;
END;
```

- Rezultat:

```
Au fost modificate 4 linii
Found TRUE
NotFound FALSE
```

Cursori expliciti

- Aşa cum am menţionat paşii de lucru sunt:

1. Declararea cursorului
2. Deschiderea cursorului
3. Încărcarea liniei curente (este mereu o cerere SELECT!)

Declarare

- Sintaxa declaraţiei unui cursor este:

```
DECLARE
  CURSOR nume_cursor IS cerere_select;
```

- unde:
  - **nume_cursor** este un identificator PL/SQL care denumeşte cursorul.
  - **cerere_select** este o cerere SELECT care nu trebuie să conţină clauza INTO. Variabila/lista de variabile în care se încarcă linia curentă din rezultat va fi specificată în instrucţiunea FETCH.

Deschidere

- Prin execuţia instrucţiunii OPEN se 'deschide' un cursor.
- Efectul practic este următorul:
  - se execută cererea SQL asociată cursorului.
  - se aduce în zona de memorie a acestuia rezultatul cererii (o tabelă rezultat).
  - se setează înregistrarea (linia) curentă la prima linie.

Deschidere

- Sintaxa instrucţiunii este:

```
OPEN nume_cursor;
```

Observaţie:

- Dacă cererea SQL nu întoarce nici o linie sau întoarce mai multe linii **nu** se ridică excepţiile NO_DATA_FOUND respectiv TOO_MANY_ROWS!

Incarcare linie

- Rezultatul lui OPEN este o tabelă rezultat iar indicatorul liniei curente este setat la prima.
- Prin instrucţiunea FETCH:
  - Se încarcă linia curentă într-o mulţime de variabile sau o singură variabilă înregistrare (având mai multe câmpuri). Corespondenţa este poziţională
  - Se incrementează indicatorul liniei curente.
  - Dacă nu s-a putut încărca o linie (mai sunt linii în rezultat) %FOUND întoarce FALSE.

Incarcare linie

- Sintaxa instrucţiunii este:

```
FETCH nume_cursor INTO lista_variabile |
  variabila_inregistrare
```

FETCH lista

- În cazul în care încărcarea se face într-o listă de variabile trebuie ca:
  - numărul de variabile din listă să fie acelaşi cu numărul de coloane din rezultatul cererii SELECT
  - tipurile variabilelor din listă să fie aceleaşi cu tipurile coloanelor din rezultatul cererii SELECT

FETCH inregistrare

- În cazul în care încărcarea se face într-o variabilă înregistrare trebuie ca:
  - numărul de câmpuri al variabilei să fie acelaşi cu numărul de coloane din rezultatul cererii SELECT
  - tipurile câmpurilor variabilei să fie aceleaşi cu tipurile coloanelor din rezultatul cererii SELECT
- Se poate defini o variabilă de acelaşi tip cu liniile rezultatului folosind construcţia **nume_cursor%rowtype**.

Atribute cursor

- Atributele cursorului sunt ca niste functii care pot fi consultate după ce s-a făcut primul FETCH şi ele au următoarele valori:

1. **`nume_cursor%ROWCOUNT`**: numărul de linii încărcate cu FETCH de la deschiderea cursorului.
2. **`nume_cursor%FOUND`**: TRUE dacă ultimul FETCH a încărcat o nouă linie şi FALSE altfel.

Atribute cursor

- Atributele cursorului ...

3. **`nume_cursor%NOTFOUND`**: TRUE dacă ultimul FETCH nu a încărcat o nouă linie şi FALSE altfel.
4. **`nume_cursor%ISOPEN`**: TRUE dacă acel cursor e deschis şi FALSE altfel

- Notă: In tabelul urmator expresia 'ultimul FETCH' denumeşte execuţia care nu mai întoarce o linie de rezultat.

Valori atribute

Folosit		%FOUND	%ISOPEN	%NOTFOUND	%ROWCOUNT
înainte de	OPEN	excepţie	FALSE	excepţie	excepţie
după	OPEN	NULL	TRUE	NULL	0
înainte de	Primul FETCH	NULL	TRUE	NULL	0
după	Primul FETCH	TRUE	TRUE	FALSE	1
înainte de	Următoarele FETCH	TRUE	TRUE	FALSE	1
după	Următoarele FETCH	TRUE	TRUE	FALSE	depinde de date
înainte de	Ultimul FETCH	TRUE	TRUE	FALSE	depinde de date
după	Ultimul FETCH	FALSE	TRUE	TRUE	depinde de date
înainte de	CLOSE	FALSE	TRUE	TRUE	depinde de date
după	CLOSE	excepţie	FALSE	excepţie	excepţie

Exemplul 1

- Listarea numelor studenţilor de la specializarea cu cod 21.
- Parcurgerea rezultatului se face cu un ciclu LOOP şi ieşirea este determinată de atributul %NOTFOUND:

```
DECLARE
  -- declarare cursor
 CURSOR studenti IS SELECT * FROM STUD WHERE CODS = 21;
  -- variabila pentru incarcare linie
 v_stud  studenti%rowtype;
BEGIN
-- deschidere cursor
OPEN studenti;
```

Exemplul 1

```
LOOP
   -- incarcarea liniei curente
   FETCH studenti INTO v_stud;
   -- iesire cand nu mai sunt linii
   EXIT WHEN studenti%notfound;
   dbms_output.put_line(studenti%rowcount||
                        ' Student:
   '||v_stud.nume);
END LOOP;
-- inchidere cursor
CLOSE studenti;
END;
```

Exemplul 1

- Rezultatul execuţiei este:

```
1 Student: ION
2 Student: STANCA
3 Student: ALEX
4 Student: ELENA
```

Exemplul 2

- Exemplul 2: Rezultate returnate de ISOPEN, FOUND

```
DECLARE
 CURSOR studenti IS SELECT * FROM STUD WHERE CODS
  = 21;
 v_stud studenti%rowtype;
BEGIN
-- inca nu e deschis
IF studenti%isopen THEN
  dbms_output.put_line('Cursorul e deschis');
ELSE
  dbms_output.put_line('Cursorul nu e deschis');
END IF;
```

Exemplul 2

```
-- se deschide cursorul
OPEN studenti;
IF studenti%isopen THEN
  dbms_output.put_line('Cursorul e deschis');
ELSE
  dbms_output.put_line('Cursorul nu e deschis');
END IF;
-- este deschis, se incarca o linie
FETCH studenti INTO v_stud;
IF studenti%found THEN
  dbms_output.put_line('Exista rezultat');
ELSE
  dbms_output.put_line('Nici macar o linie');
END IF;
```

Exemplul 2

```
-- inchidere cursor
CLOSE studenti;
IF studenti%isopen THEN
   dbms_output.put_line('Cursorul e
  deschis');
ELSE
   dbms_output.put_line('Cursorul nu e
  deschis');
END IF;
END;
```

Exemplul 2

- Rezultatul execuţiei blocului de mai sus este:

```
Cursorul nu e deschis
Cursorul e deschis
Exista rezultat
Cursorul nu e deschis
```

Ciclul FOR pentru cursori

- Pentru a uşura parcurgerea liniilor tabelei rezultat asociată unui cursor deschis, în PL/SQL există un ciclu FOR special.
- Sintaxa lui este următoarea:

```
FOR v_inregistrare IN cursor LOOP
   instructiuni;
END LOOP
```

FOR pentru cursori

- Observaţii:
  - Variabila înregistrare nu trebuie definită de utilizator, acest lucru făcându-se automat.
  - Domeniul de valabilitate al acestei variabile este corpul ciclului.
  - OPEN, FETCH şi CLOSE se execută automat.
  - Se execută câte un pas al ciclului pentru fiecare linie a rezultatului.
  - Ieşirea din ciclu se face automat la terminarea liniilor din rezultat.

Exemplu

```
DECLARE
  CURSOR studenti IS
    SELECT * FROM STUD WHERE CODS = 21;
BEGIN
FOR v_stud IN studenti LOOP
  -- ciclul deschide cursorul,
  -- v_stud nu e declarata
  -- nu e necesar FETCH
  dbms_output.put_line(studenti%rowcount||
        ' Student: '||v_stud.nume);
END LOOP;  -- inchide automat cursorul
END;
```

FOR cu cursori impliciti

- Acest tip de ciclu FOR se poate folosi şi pentru **cursori impliciţi**, ca în exemplul de mai jos:

```
BEGIN
-- nu e necesara declararea cursorului si
-- nici a variabilei v_stud
FOR v_stud IN (select * from stud where cods=21) LOOP
  -- nu e necesar FETCH
  dbms_output.put_line('Student: ' ||v_stud.nume);
END LOOP;
-- nu e necesara inchiderea cursorului
END;
```

- În acest caz returnarea a 0 sau mai multor linii nu duce la ridicarea de excepţii!

Cursori cu parametri

- Unui cursor i se pot transmite parametri. În acest caz:
- La declarare se specifică parametrii formali şi tipul lor (fără să se specifice dimensiunea, deci VARCHAR2 şi nu VARCHAR2(10) de exemplu).
- Aceşti parametri sunt folosiţi în cererea SQL asociată.
- La deschidere se specifică parametrii actuali.
- Sintaxa declarării unui parametru este:

```
parametru [IN] tipdedate [{:= | DEFAULT}
  expresie]
```

Exemplu

```
DECLARE
  CURSOR studenti (v_cods number :=21,
                   v_loc varchar2:='BUCURESTI') IS
  SELECT nume FROM stud
  WHERE cods = v_cods and loc = v_loc;
  i number;
BEGIN
FOR i IN 1..5 LOOP -- cod specializare = i*5+1
 dbms_output.put_line('Specializarea ' || (i*5+1));
 FOR v_stud in studenti((i*5+1), 'PLOIESTI') LOOP
    dbms_output.put_line(studenti%rowcount||
                         ' Student: '||v_stud.nume);
 END LOOP;  -- inchide automat cursorul
END LOOP; -- ciclul FOR i in 1..5
```

Exemplu

```
dbms_output.put_line('Cu parametrii impliciti');
FOR v_stud IN studenti LOOP
   dbms_output.put_line(studenti%rowcount|| 'Student: '
   ||v_stud.nume);
END LOOP;  -- inchide automat cursorul
END;
```

Exemplu

- Rezultatul este:

```
Specializarea 6
Specializarea 11
1 Student: MARIA
Specializarea 16
Specializarea 21
1 Student: ION
Specializarea 26
Cu parametrii impliciti
1 Student: STANCA
2 Student: ELENA
```

Cursori cu subcereri

- În clauza FROM a cererii SQL care defineşte un cursor poate să apară o subcerere SQL.
- Ea se evaluează o singură dată şi rezultatul său este folosit pentru calculul rezultatului cererii globale.
- Exemplu: declaraţia cursorului conţine pe clauza FROM o subcerere SQL care returnează o tabelă cu trei coloane conţinând numărul şi numele specializării precum şi numărul de studenţi pentru specializările cu mai mult de 3 studenţi:

Cursori cu subcereri

```
DECLARE
CURSOR s IS SELECT t1.cods, t1.nume, cati
FROM SPEC t1, (SELECT cods, count(*) cati
               FROM stud GROUP BY cods) t2
WHERE t1.cods = t2.cods AND cati > 3;
BEGIN
FOR spc IN s LOOP
  dbms_output.put_line(s%rowcount||
    ' Specializarea: '|| spc.cods||          '
  '||spc.nume|| ' ' ||spc.cati);
END LOOP;
END;
```

Cursori cu subcereri

- Rezultatul va fi:

```
1 Specializarea: 21 GEOGRAFIE 4
2 Specializarea: 24 ISTORIE 5
```

- Bineînţeles subcererile pot fi prezente şi pe clauza WHERE, fie corelate, fie necorelate.

Referinte catre cursori

- Aceste referinţe (de tip REF CURSOR) sunt similare pointerilor din limbajele C sau Pascal.
- Definirea se face cu:

```
TYPE nume_tip_pointer IS REF CURSOR
                  [RETURN tip_inregistrare];
v_pointer_cursor nume_tip_pointer;
```

Referinte catre cursori

- Clauza RETURN se foloseşte în definirea pachetelor (package) pentru a specifica tipul de date întors de cursor.
- Deschiderea cursorului se face în acest caz cu:
- OPEN v_pointer_cursor FOR cerere_sql;
- Deci asocierea cererii SQL se face nu la declarare ci la deschiderea cursorului.

Exemplu

```
DECLARE
TYPE Referinta IS REF CURSOR RETURN stud%rowtype;
c_ref Referinta;  -- variabila de tip REF CURSOR
s stud%rowtype;   -- variabila pentru incarcarea
                  -- unei linii
BEGIN
-- deschidere cursor cu specificarea cererii
OPEN c_ref FOR select * from stud where cods=11;
LOOP
  FETCH c_ref INTO s;
  EXIT WHEN c_ref%notfound;
  dbms_output.put_line(c_ref%rowcount||
     ' Nume: '|| s.nume || ' Punctaj '|| s.punctaj);
END LOOP;
CLOSE c_ref;
```

Exemplu

```
-- deschidere cursor cu alta cerere SQL
OPEN c_ref FOR select * from stud where loc='BRASOV';
LOOP
  FETCH c_ref INTO s;
  EXIT WHEN c_ref%notfound;
  dbms_output.put_line(c_ref%rowcount||
     ' Nume: '|| s.nume|| ' Localitate '|| s.loc);
END LOOP;
END;
```

Exemplu

- Rezultatul va fi:

```
1 Nume: GEORGE Punctaj 2890
2 Nume: VASILE Punctaj 390
3 Nume: MARIA Punctaj 1400
1 Nume: ALEX Localitate BRASOV
2 Nume: FLOREA Localitate BRASOV
3 Nume: VOICU Localitate BRASOV
```

Restrictii REF CURSOR

- Printre restricţiile privind variabilele REF CURSOR menţionăm:
  - Nu se pot declara în pachete (package)
  - Nu se pot compara pentru egalitate, inegalitate sau valoare nulă
  - Nu li se poate asigna o valoare nulă

Restrictii REF CURSOR

- Restrictii… :
- Tipul REF CURSOR nu poate fi folosit în SQL la definirea coloanelor unei tabele sau vederi.
- Nu poate fi tipul elementelor unei colecţii
- Nu sunt interschimbabile cu variabilele tip cursor: de exemplu nu le putem folosi în cicluri FOR pentru cursori (mai sus am folosit LOOP).

Excepţia ROWTYPE_MISMATCH

- Când se încearcă încărcarea unei linii într-o variabilă care nu corespunde ca tip se ridică excepţia ROWTYPE_MISMATCH.
- În exemplul de mai jos se încearcă încărcarea unei linii cu mai multe valori
- într-o variabilă scalară. În acest caz se va ridica excepţia menţionată:

```
DECLARE
TYPE Referinta IS REF CURSOR;
c_ref Referinta;  -- variabila de tip REF CURSOR
s varchar2(10);   -- variabila pentru incarcarea
                  -- unei linii. NU E CORECTA!
```

Excepţia ROWTYPE_MISMATCH

```
BEGIN
-- deschidere cursor cu specificarea cererii
OPEN c_ref FOR select * from stud where cods=11;
LOOP
  FETCH c_ref INTO s;   -- aici exceptie!
  EXIT WHEN c_ref%notfound;
END LOOP;
CLOSE c_ref;
EXCEPTION
  WHEN rowtype_mismatch THEN
    dbms_output.put_line('Exceptie
  rowtype_mismatch');
END;
```

SELECT . . FOR UPDATE

- În cazul în care prin intermediul unui cursor se execută operaţii de actualizare a tabelelor se poate folosi clauza FOR UPDATE pentru a bloca liniile necesare în momentul în care se deschide cursorul
- Implicit Oracle blochează o linie în momentul în care aceasta e actualizată.
- De asemenea, când se foloseşte CURRENT OF pentru a modifica liniile unei tabele acestea trebuie blocate înainte cu FOR UPDATE.

SELECT . . FOR UPDATE

- Sintaxa este:

```
DECLARE CURSOR nume_cursor [(parametri)] IS
  SELECT . . .
  . . .
  FOR UPDATE [OF coloana] [NOWAIT]
```

SELECT . . FOR UPDATE

- În momentul deschiderii cursorului liniile găsite de cererea SELECT sunt blocate.
- Ele sunt deblocate la sfârşitul tranzacţiei (COMMIT). După ce modificările au fost comise nu se mai pot încărca linii dintr-un cursor definit cu FOR UPDATE.
- NOWAIT specifică faptul că dacă liniile respective sunt blocate în acel moment Oracle să nu aştepte (aceasta fiind acţiunea standard).
- Exemplu: liniile tabelei STUD sunt blocate pentru ca pe parcursul actualizării să nu fie modificate de alţi utilizatori:

Exemplu

```
DECLARE
CURSOR c_stud IS select nume, loc, rowid
                 from stud FOR UPDATE NOWAIT;
v_nume stud.nume%type; v_loc stud.loc%type;
v_rowid rowid;
v_nr number;
BEGIN
OPEN c_stud; v_nr := 0;
LOOP
  FETCH c_stud into v_nume, v_loc, v_rowid;
  EXIT WHEN c_stud%notfound;
  dbms_output.put_line(v_nume||' '|| v_loc);
```

Exemplu

```
UPDATE stud SET punctaj = punctaj * 1.1
  WHERE rowid = v_rowid;
  -- aici e recomandat CURRENT OF c_stud
  v_nr := v_nr + 1;
END LOOP;
dbms_output.put_line('Actualizate: '||v_nr);
CLOSE c_stud;
COMMIT;
EXCEPTION
  WHEN others THEN
    dbms_output.put_line(SQLCODE|| ' ' ||
      SQLERRM);
END;
```

Exemplu

1. În cazul în care există linii blocate de alţi utilizatori se va genera o excepţie (din cauza lui NOWAIT).

- Mesajul va fi:

```
-54 ORA-00054: resource busy and acquire with NOWAIT
  specified
```

2. Dacă liniile nu sunt blocare de alt utilizator rezultatul este:

```
GEORGE BUCURESTI
VASILE PITESTI
. . . . . .
MARIUS PITESTI
VOICU BRASOV
Actualizate: 12
```

SELECT . . FOR UPDATE

- De asemenea prin clauza FOR UPDATE se blochează linii dintr-o tabelă aflată pe clauza FROM a unei cereri SQL multi-tabelă.
- Pentru aceasta trebuie specificată coloana care se doreşte modificată:

```
FOR UPDATE OF coloana
```

Exemplu

```
DECLARE
CURSOR c_stud IS
  select st.nume, st.rowid, sp.nume
  from stud st,spec sp
  where st.cods=sp.cods and
        sp.nume='MATEMATICA'
  FOR UPDATE of punctaj NOWAIT;
v_stnume stud.nume%type;
v_spnume spec.nume%type;
v_rowid rowid; v_nr number;
```

Exemplu

```
BEGIN
OPEN c_stud;
v_nr := 0;
LOOP
  FETCH c_stud INTO v_stnume, v_rowid, v_spnume;
  EXIT WHEN c_stud%notfound;
  dbms_output.put_line(v_stnume||' '||v_spnume);
  UPDATE stud SET punctaj = punctaj * 1.1
  WHERE rowid = v_rowid;
  -- aici e recomandat CURRENT OF c_stud
    v_nr := v_nr + 1;
END LOOP;
dbms_output.put_line('Actualizate: '||v_nr);
```

Exemplu

```
CLOSE c_stud;
COMMIT;
EXCEPTION
   WHEN others THEN
      dbms_output.put_line(SQLCODE||' '||SQLERRM);
END;
```

- Rezultat:

```
GEORGE MATEMATICA
VASILE MATEMATICA
MARIA MATEMATICA
Actualizate: 3
```

WHERE CURRENT OF

- În cazul în care folosim FOR UPDATE linia curentă a unui cursor se poate identifica şi prin construcţia folosită în WHERE :

```
CURRENT OF nume_cursor
```

- Exemplu:

```
DECLARE
CURSOR c_stud IS select nume, loc
                 from stud FOR UPDATE;
v_nume stud.nume%type; v_loc stud.loc%type;
v_nr number;
```

Exemplu

```
BEGIN
OPEN c_stud; v_nr := 0;
LOOP
   FETCH c_stud INTO v_nume, v_loc;
   EXIT WHEN c_stud%notfound;
   dbms_output.put_line(v_nume||' '||v_loc);
   UPDATE stud1
   SET punctaj = punctaj * 1.1
   WHERE CURRENT OF c_stud;  -- folosire
   v_nr := v_nr + 1;
END LOOP;
```

Exemplu

```
COMMIT;
dbms_output.put_line('Actualizate: ' ||v_nr);
CLOSE c_stud;
EXCEPTION
  WHEN others THEN
    dbms_output.put_line(SQLCODE||
     ' ' ||SQLERRM);
END;
```

Exemplu

- Rezultatul va fi:

```
GEORGE BUCURESTI
VASILE PITESTI
MARIA PLOIESTI
ION PLOIESTI
STANCA BUCURESTI
ALEX BRASOV
ELENA BUCURESTI
ADRIAN BUCURESTI
FLOREA BRASOV
OANA BUCURESTI
MARIUS PITESTI
VOICU BRASOV
Actualizate: 12
```

Administrarea bazelor de date

CURS 5: TABLE şi VARRAY

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Asemanari

- TABLE si VARRAY (ambele se mai numesc si colectii) se aseamănă între ele în următoarele privinţe:

1. O colecţie de acest fel poate fi stocată într-o celulă a unei tabele din baza de date (deci o tabelă poate avea o coloană de tip colecţie TABLE sau VARRAY). În cazul tablourilor asociative (TABLE .. INDEX BY) acest lucru nu era posibil.
2. Accesul la elemente se face prin indicele acestora care porneşte de la 1.
3. În ambele cazuri variabilele de acest tip pot fi iniţializate la declarare şi li se pot atribui valori în porţiunea executabilă folosind constructori de aceeaşi formă.

Asemanari

- TABLE si VARRAY se aseamănă...

4. Numărul de elemente dintr-o astfel de colecţie poate varia pe parcursul execuţiei programului (dar în cazul VARRAY nu poate depăşi limita maximă declarată)
5. Elementele unei astfel de colecţii sunt toate de acelaşi tip de date.
6. Limitările privind tipul elementelor sunt aceleaşi în cele două cazuri.

Deosebiri

1. Numărul maxim de elemente ale tipului TABLE nu este fixat la descrierea lui, spre deosebire de VARRAY unde se specifică această limită.
2. Indicii elementelor unui VARRAY se păstrează prin stocarea în baza de date şi regăsirea lui pe când în cazul TABLE acest lucru nu este adevărat.

- Din acest motiv tipul TABLE seamănă cu mulţimile din limbajele de programare Pascal şi C spre deosebire de tipul VARRAY care este asemănător cu tablourile.

Deosebiri

3. În cazul unui VARRAY indicii elementelor sunt succesivi, pornind de la indicele 1, crescător cu o unitate.
- La TABLE deşi intiţial indicii elementelor sunt succesivi, prin ştergerea unor elemente pot să apară 'goluri' (operaţia de ştergere 'din mijloc' nu este posibilă la VARRAY).

3. Ca stocare fizică în baza de date, datele de tip VARRAY se memorează fie în interiorul tabelei (pentru dimensiune mai mică de 4 KB) fie în spaţiul asociat tabelei (tablespace).
4. În cazul TABLE stocarea valorilor se face într-o aşa numită 'tabelă de stocare' (store table) asociată tabelei de bază.

Limitari

- În cazul declarării lor în PL/SQL tipul elementelor poate fi oricare tip valid PL/SQL cu excepţia REF CURSOR
- În cazul folosirii lor în SQL, pe lângă REF CURSOR sunt interzise şi tipurile:
  - BINARY_INTEGER, PLS_INTEGER
  - BOOLEAN
  - LONG, LONG RAW
  - NATURAL, NATURALN
  - POSITIVE, POSITIVEN
  - REF CURSOR
  - SIGNTYPE
  - STRING

Declarare

- Sintaxa definiţiei acestor tipuri este următoarea:
  - TABLE:

```
TYPE nume_tip IS TABLE OF tip_element [NOT NULL];
```

  - VARRAY:

```
TYPE nume_tip IS {VARRAY | VARYING ARRAY}
(indice_maxim) OF tip_element [NOT NULL];
```

unde:

- **nume_tip** - Numele tipului definit de acea declaraţie
- **indice_maxim** - Dimensiunea maximă pentru acel VARRAY
- **tip_element** - tipul elementelor colecţiei
- **NOT NULL** - specifică opţional că elementele nu pot fi nule

Exemple

- TABLE: Declararea a două colecţii care pot conţine un număr nedefinit de denumiri de cursuri la care sunt înscrişi doi studenţi:

```
TYPE ListaCursuri IS TABLE OF
  VARCHAR2(30);
ListaMea ListaCursuri;
ListaLui ListaMea%Type;
```

Exemple

- VARRAY: Declararea a două colecţii care pot conţine maxim 25 de denumiri de cursuri la care sunt înscrişi doi studenţi:

```
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri;
ListaLui ListaMea%Type;
```

- Se observă că se poate folosi construcţia %TYPE pentru a defini o variabilă colecţie ca având acelaşi tip cu altă variabilă colecţie.

%TYPE şi %ROWTYPE

- Pentru tipul elementelor se pot folosi de asemenea construcţiile %TYPE şi %ROWTYPE:

```
TYPE NumeStud IS TABLE OF stud.nume%TYPE;
TYPE ListaStud IS TABLE OF stud%ROWTYPE;
CURSOR c_spec IS SELECT * FROM spec;
TYPE ListaSpec IS VARRAY(20) OF
  c_spec%ROWTYPE;
```

- În primul caz, NumeStud este tipul unor colecţii de valori scalare de acelaşi tip cu coloana NUME din tabela STUD.
- În celelalte două cazuri ListaStud şi ListaSpec sunt colecţii având ca elemente înregistrări definite pe baza structurii tabelei stud respectiv a cursorului c_spec.

Elemente de tip înregistrare

- Elementele unor colecţii de acest tip pot fi şi înregistrări definite de utilizator:

```
DECLARE
TYPE Student IS RECORD (
  nume VARCHAR2(40),
  varsta DATE);
TYPE grupa IS VARRAY(25) OF Student;
TYPE seria IS TABLE OF Student;
```

Iniţializarea la declarare

- Iniţializarea unor colecţii de acest tip se poate face la declarare prin construcţia:

```
:= nume_tip_colectie(lista_de_valori)
```

- Exemple:

```
TYPE ListaCursuri IS TABLE OF
  VARCHAR2(30);
ListaMea ListaCursuri :=
  ListaCursuri('Analiza', 'Algebra',
  'Fizica', 'Limba engleza');
```

Iniţializarea la declarare

- sau similar în cazul VARRAY:

```
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri :=
  ListaCursuri('Analiza', 'Algebra',
  'Fizica', 'Limba engleza');
```

Iniţializarea la declarare

- În cazul în care constructorul a fost apelat fără valori între paranteze se generează o colecţie ***vidă dar nenulă***:

```
DECLARE
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri := ListaCursuri();
BEGIN
   IF ListaMea IS NOT NULL THEN
     .   .   .   -- condiţia adevarata!
```

Iniţializarea la declarare

- În schimb dacă o colecţie nu este iniţializată testul de NOT NULL returnează FALSE (deci IS NULL returnează TRUE):

```
DECLARE
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
   IF ListaMea IS NOT NULL THEN
     .   .   .   -- conditie falsa
```

Atribuirea

- Se poate lăsa neiniţializată colecţia la declarare şi iniţializa în zona executabilă. Exemplul următor este doar pentru TABLE dar similar se procedează pentru VARRAY:
- Exemplu:

```
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza',
        'Algebra', 'Fizica', 'Limba
 engleza');
```

Atribuirea

- Lista de valori poate conţine valori nule:

```
TYPE ListaCursuri IS VARRAY(25) OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza', NULL, NULL,
        'Algebra', NULL, 'Fizica', 'Limba
  engleza');
```

- În ultimele două exemple au fost asignate colecţii conţinând 4 respectiv 7 elemente (au fost numărate şi valorile NULL).

Atribuiri intre colectii

- Două colecţii **de acelaşi tip** se pot atribui una alteia. Dacă însă tipul diferă rezultă o excepţie.

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
TYPE AltTip IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
ListaLui ListaMea%Type;
ListaEi  AltTip;
BEGIN
  ListaLui := ListaCursuri('Analiza', NULL, NULL,
         'Algebra', NULL, 'Fizica', 'Limba engleza');
  ListaMea := ListaLui;
  ListaEi := ListaMea; -- va genera exceptie
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count);
END;
```

Compararea colecţiilor

- O colecţie poate fi testată dacă este nulă sau nu cu IS NULL şi IS NOT NULL.
- În schimb nu putem folosi = sau <> pentru a testa egalitatea sau inegalitatea a două colecţii.
- Din acest motiv o coloană care este de tip colecţie nu poate apărea în cereri SQL în conjuncţie cu clauzele DISTINCT, GROUP BY sau ORDER BY.

Metode pentru TABLE şi VARRAY

- În manipularea datelor de tip TABLE şi VARRAY putem folosi metodele menţionate anterior: EXISTS, COUNT, FIRST, LAST, PRIOR, NEXT şi DELETE.
- Metoda DELETE nu se poate aplica la VARRAY. Se foloseşte TRIM pentru a şterge elemente de la sfârşitul tabloului.

Metode pentru TABLE şi VARRAY

- În plus se pot folosi:

LIMIT	**Doar pentru VARRAY: numărul maxim de elemente permis. Pentru celelalte tipuri întoarce NULL.**
EXTEND[(n[,i)]]	**Extinde tabloul cu un element nul, *n* elemente nule sau n elemente egale cu elementul cu indice *i*.**
TRIM[(n)]	**Şterge un element, respectiv *n* elemente de la sfârşitul tabloului.**

Exemplu: Table

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza', NULL, NULL, 'Algebra', NULL,
  'Fizica', 'Limba engleza');
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 7
  ListaMea.Extend(3, 2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 10
  ListaMea.Delete(7);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 9
  ListaMea.Delete(3, 6);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 5
  ListaMea.Trim(2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 3
END;
```

Exemplu: VARRAY

```
DECLARE
TYPE ListaCursuri IS VARRAY(25) OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza', NULL, NULL,
         'Algebra', NULL, 'Fizica', 'Limba engleza');
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count);
  -- 7
  ListaMea.Extend(3, 2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count);
  -- 10
  ListaMea.Trim(2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count);
  -- 3
END;
```

Exemple

- Metodele FIRST, LAST, NEXT şi PRIOR ne permit parcurgerea unei variabile de tip TABLE sau VARRAY.
- În unele exemple de mai jos am folosit colecţii TABLE cu elemente şterse.

Exemple

- Exemplul 1: Parcurgerea unei colecţii cu ciclu FOR. Acest bloc ridică excepţie dacă există elemente şterse.

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
i NUMBER;
BEGIN
  ListaMea := ListaCursuri('Analiza', 'Algebra', 'Sport',
 'Fizica', 'Limba engleza', 'Programare');
  FOR i IN ListaMea.FIRST .. ListaMea.LAST LOOP
    dbms_output.put_line('Curs: '||i|| ' '||ListaMea(i));
  END LOOP;
END;
```

Exemple

- Exemplul 1: Parcurgerea unei colecţii cu ciclu FOR. Acest bloc ridică excepţie dacă există elemente şterse.

```
Curs: 1 Analiza
Curs: 2 Algebra
Curs: 3 Sport
Curs: 4 Fizica
Curs: 5 Limba engleza
Curs: 6 Programare
```

Exemple

- Exemplul 2: Parcurgere folosind LOOP pentru a evita excepţia menţionată anterior:

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
i NUMBER;
BEGIN
  ListaMea := ListaCursuri('Analiza', 'Algebra', 'Sport',  'Fizica',
   'Limba engleza', 'Programare');
  ListaMea.Delete(3,4);
  i := ListaMea.FIRST;
  LOOP
    dbms_output.put_line('Curs: '||i||' '||ListaMea(i));
    i := ListaMea.NEXT(i);
    EXIT WHEN i IS NULL;
  END LOOP;
END;
```

Exemple

- Rezultat:

```
Curs: 1 Analiza
Curs: 2 Algebra
Curs: 5 Limba engleza
Curs: 6 Programare
```

Administrarea bazelor de date

CURS SUPLIMENTAR: Proceduri, functii, pachete si declansatori

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Ce sunt acestea?

- Procedurile şi funcţiile sunt denumite şi subprograme.
- Ele sunt blocuri PL/SQL având un nume, o listă de parametri şi, în cazul funcţiilor, o valoare returnată.
- Mai multe proceduri şi/sau funcţii pot fi 'împachetate' sub forma unui 'pachet' (package).

Declararea unei proceduri

- Sintaxa declarării unei proceduri este:

Doar pentru proceduri stocate

```
[CREATE [OR REPLACE]]
PROCEDURE nume_procedura[(parametru[, parametru]...)]
[AUTHID {DEFINER | CURRENT_USER}] {IS | AS}
[PRAGMA AUTONOMOUS_TRANSACTION;]
[declaratii locale]
BEGIN
instructiuni executabile sau NULL;
[EXCEPTION
tratare erori]
END [nume_procedura];
```

Declararea unei proceduri

- **[CREATE [OR REPLACE]]** - arată că procedura se defineşte ca obiect al bazei de date şi va fi stocată în aceasta. Altfel ea este parte a unui bloc PL/SQL.
- **[AUTHID {DEFINER | CURRENT_USER}]** - specifică dacă o procedură stocată se execută cu drepturile celui care a creat-o (valoare implicită) sau ale utilizatorului curent.
- **[PRAGMA AUTONOMOUS_TRANSACTION;]** - dacă se specifică această caracteristică, execuţia procedurii suspendă tranzacţia curentă care se reia după terminarea execuţiei acesteia.

Parametri formali

- Fiecare parametru formal se defineşte astfel:

```
nume_parametru [IN | OUT [NOCOPY] | IN OUT
   [NOCOPY]]
   numetip [{:= | DEFAULT} expresie]
```

- unde:

A. **IN**, **OUT** şi **IN OUT** arată că este vorba de un parametru de intrare, de ieşire sau bidirecţional. În cazul în care nu se specifică, parametrul este implicit transmis ca IN. În corpul unui subprogram nu se poate asigna o valoare unui parametru transmis cu IN.

Parametri formali

B. **NOCOPY**. În mod implicit parametrii de tip IN sunt transmişi prin referinţă iar cei OUT şi IN OUT prin valoare.

- În cazul unor parametri voluminoşi transmiterea prin valoare e consumatoare de resurse.
- Se poate specifica în acest caz (OUT, IN OUT) transmiterea prin referinţă folosind NOCOPY.

Parametri formali

C. **numetip** este un nume de tip fără specificarea dimensiunii (deci VARCHAR2 de exemplu şi nu VARCHAR2(10).

- Se poate totuşi asocia o dimensiune maximă prin definirea unui subtip care să corespundă dorinţelor noastre.

Parametri formali

- Exemplu:

```
DECLARE
SUBTYPE Char25 IS CHAR(25);
PROCEDURE cauta (persoana Char25) IS ...
```

Plasare

- Când o procedură este parte a unui alt bloc ea va fi plasată în zona DECLARE a acestuia şi poate fi apelată în zona executabilă cu parametrii actuali.
- Exemplu: procedura ***alt_tutor*** schimbă valoarea de pe coloana TUTOR a unui student.
- Ea nu execută operaţia în cazul în care studentul nu are un tutor sau când codul tutorului este identic cu al studentului:

```
DECLARE
 PROCEDURE alt_tutor (cod_stud INTEGER, nou_tutor INTEGER)
 IS
   tutor_actual number;
   fara_tutor EXCEPTION; este_el EXCEPTION;
 BEGIN
   SELECT tutor INTO tutor_actual FROM stud1
   WHERE matr = cod_stud;
```

Plasare

```
IF nou_tutor=cod_stud THEN
    RAISE este_el;
  ELSIF tutor_actual IS NULL THEN
    RAISE fara_tutor;
  ELSE
    UPDATE stud1 SET tutor = nou_tutor
    WHERE matr = cod_stud;
    dbms_output.put_line('Actualizat
'||cod_stud);
  END IF;
```

Plasare

```
EXCEPTION
   WHEN NO_DATA_FOUND THEN
     dbms_output.put_line('Nu exista studentul');
   WHEN fara_tutor THEN
     dbms_output.put_line('Nu are tutor');
   WHEN este_el THEN
     dbms_output.put_line('Nu isi poate fi tutor');
  END alt_tutor;
BEGIN  -- blocul principal
  alt_tutor(1325, 1645);
  alt_tutor(1456, 1645);
  alt_tutor(1645, 1645);
END;
Rezultatul va fi:
Actualizat 1325
Nu are tutor
Nu isi poate fi tutor
```

Declararea unei funcţii

- Sintaxa simplificată a declaraţiei unei funcţii este următoarea:

Doar pentru functii stocate

```
[CREATE [OR REPLACE ] ]
FUNCTION nume_functie [ ( parametru [ , parametru ]... )]
 RETURN tip_date_returnate
 [ AUTHID { DEFINER | CURRENT_USER } ]
{IS | AS}
[ PRAGMA AUTONOMOUS_TRANSACTION; ]
[declaratii locale]
BEGIN
instructiuni executabile sau NULL;
[EXCEPTION
tratare erori]
END [nume_functie];
```

Declararea unei funcţii

- Elementele care apar şi la proceduri au aceeaşi semnificaţie.
- În plus apare tipul valorii returnate care, ca şi parametrii formali, nu poate fi constrâns prin specificarea unei dimensiuni maxime.
- În zona executabilă trebuie să apară instrucţiunea **RETURN expresie** care specifică valoarea returnată de funcţie.

Exemplu

- Programul de schimbare tutor rescris prin transformarea procedurii într-o funcţie care întoarce un şir de caractere:

```
DECLARE
 FUNCTION alt_tutor (cod_stud INTEGER, nou_tutor INTEGER)
 RETURN VARCHAR2 IS
   tutor_actual number;
   fara_tutor EXCEPTION;
   este_el EXCEPTION;
 BEGIN
   SELECT tutor INTO tutor_actual FROM stud1
   WHERE matr = cod_stud;
```

Exemplu

```
  IF nou_tutor=cod_stud THEN
    RAISE este_el;
  ELSIF tutor_actual IS NULL THEN
    RAISE fara_tutor;
  ELSE
    UPDATE stud1 SET tutor = nou_tutor
    WHERE matr = cod_stud;
    RETURN 'Actualizat '||cod_stud;
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 'Nu exista studentul';
  WHEN fara_tutor THEN
    RETURN 'Nu are tutor';
  WHEN este_el THEN
    RETURN 'Nu isi poate fi tutor';
 END alt_tutor;
```

Exemplu

```
BEGIN  -- blocul principal
  dbms_output.put_line(alt_tutor(1325, 1645));
  dbms_output.put_line(alt_tutor(1456, 1645));
  dbms_output.put_line(alt_tutor(1645, 1645));
END;
```

- După cum se poate observa RETURN poate fi prezent şi în zona de tratare a excepţiilor.

Apelarea unei funcţii în SQL

- Pentru ca o funcţie stocată să fie apelabilă în cereri SQL trebuie să respecte următoarele restricţii:

1. Când este apelată într-o cerere SELECT funcţia nu trebuie să modifice nimic în tabelele bazei de date.
2. Când este apelată în INSERT, UPDATE sau DELETE funcţia nu poate regăsi sau modifica tabela actualizată de cererea SQL.

Apelarea unei funcţii în SQL

3. Când este apelată din SELECT, INSERT, UPDATE sau DELETE funcţia nu poate executa:
   - cereri de control pentru tranzacţii (ex.: COMMIT),
   - cereri de control pentru sesiunea de lucru (ex.: SET ROLE) sau
   - cereri de control sistem (ca ALTER SYSTEM).
   - De asemenea nu poate executa cereri care se comit automat (ca de exemplu CREATE).

Parametrii unui subprogram: IN

Modul IN este modul implicit de transmitere.

- Parametrul este un parametru de intrare.
- Parametrul este transmis prin referinţă, deci se transmite o adresă a originalului.
- Parametrul acţionează ca o constantă: în corpul subprogramului acesta nu poate fi modificat (de exemplu prin atribuire sau SELECT INTO), o astfel de operaţie ducând la erori de compilare.
- La apelarea subprogramului parametrul actual poate fi o constantă, un literal, o variabilă iniţializată sau o expresie.

Parametrii unui subprogram: IN

- Spre deosebire de celelalte două moduri pentru aceşti parametri se pot defini valori implicite (DEFAULT).
- Specificarea unei valori implicite se face ca în exemplul următor:

```
FUNCTION marire_bursa(
cod_stud INTEGER,
suma REAL DEFAULT 100) RETURN VARCHAR2
  IS . . .
```

- În acest caz funcţia se poate chema cu 2 parametri actuali dar şi cu unul singur, în al doilea caz al doilea având valoarea implicită 100:

```
marire_bursa(1645, 200)
marire_bursa(1645)
```

Parametrii unui subprogram: IN

- În cazul următor însă:

```
FUNCTION marire_bursa(
suma REAL DEFAULT 100,
cod_stud INTEGER) RETURN VARCHAR2
   IS . . .
```

- Apelurile urmatoare semnalează eroare şi nu actualizează bursa studentului cu matricola 1645.

```
marire_bursa(1645)
marire_bursa(, 1645)
```

- Rezolvarea acestor tipuri de situaţii este prezentată la sfârşitul acestui subcapitol.

Parametrii unui subprogram: OUT

- Parametrul este un parametru de ieşire.
- Parametrul este transmis prin valoare, deci se transmite o copie a originalului.
- Parametrul poate fi modificat în corpul subprogramului.
- La apelarea subprogramului parametrul actual trebuie să fie o variabilă.
- La apelare acesta poate avea o valoare care se pierde însă la apel (în afara cazului când se specifică NOCOPY).

Parametrii unui subprogram: OUT

- În momentul apelului parametrul este iniţializat cu NULL. Din acest motiv tipul lui nu poate fi unul constrâns de NOT NULL (ex.: POSITIVEN, NATURALN)
- Parametrului nu i se poate asocia o valoare implicită (DEFAULT).
- În corpul subprogramului trebuie să i se asocieze o valoare (altfel rămâne cu valoarea NULL).
- La ieşirea din subprogram PL/SQL asignează valoarea respectivă (inclusiv NULL) parametrului actual.
- Dacă subprogramul iese cu o excepţie netratată asignarea de mai sus nu este făcută!

Parametrii unui subprogram: INOUT

- Parametrul este un parametru de intrare şi de ieşire.
- Parametrul este transmis prin valoare, deci se transmite o copie a originalului.
- Parametrul poate fi modificat în corpul subprogramului.
- La apelarea subprogramului parametrul actual trebuie să fie o variabilă iniţializată care se poate folosi în corpul subprogramului.

Parametrii unui subprogram: INOUT

- Parametrului nu i se poate asocia o valoare implicită (DEFAULT).
- În corpul subprogramului parametrului i se poate asocia o valoare (altfel aceasta rămâne valoarea de la apel).
- La ieşirea din subprogram PL/SQL asignează valoarea respectivă (inclusiv NULL) parametrului actual.
- Daca subprogramul iese cu o excepţie netratată asignarea de mai sus nu este făcută!

Opţiunea NOCOPY

- Se aplică pentru parametrii OUT sau IN OUT specificând că transmiterea se face prin referinţă nu prin valoare.
- În unele cazuri ea este ignorată de către sistem, deci transmiterea se face tot prin valoare.
- Protocolul RPC nu permite decât transmiterea prin valoare. În cazul mutării unei proceduri având parametri NOCOPY pe alt host automat la folosirea ei aceştia vor fi transmişi prin valoare.

Opţiunea NOCOPY

- În cazul ieşirii din subprogram cu excepţie netratată nu se poate şti exact starea parametrului actual:
  - dacă s-a transmis totuşi prin referinţă modificările făcute până la apariţia excepţiei se reflectă în valoarea parametrului actual
  - dacă transmiterea s-a făcut prin valoare PL/SQL nu copiază valorile în parametrul actual.

Specificarea valorilor parametrilor

- Există posibilitatea ca la apelul unui subprogram parametrii actuali să fie în altă ordine decât cei formali. În acest caz se specifică pentru parametrii formali care sunt parametrii actuali asociaţi:
- Exemplu. Funcţia:

```
FUNCTION marire_bursa(
suma REAL DEFAULT 100,
cod_stud INTEGER) RETURN VARCHAR2
  IS . . . .
```

- poate fi apelată în următoarele moduri:

```
marire_bursa(200, 1645); -- pozitional
marire_bursa(suma => 200, cod_stud => 1645); -- cu nume
marire_bursa(cod_stud => 1645, suma => 200); -- cu nume
marire_bursa(cod_stud => 1645);  -- cu nume; suma = 100
marire_bursa(200, cod_stud => 1645); -- mixt
```

DROP

- Ştergerea unei proceduri sau funcţii se face cu DROP:
- Sintaxa:

```
DROP PROCEDURE nume_procedura;
DROP FUNCTION nume_functie;
```

- Exemple:

```
DROP FUNCTION marire_bursa;
DROP PROCEDURE alt_tutor;
```

Pachete

- Mai multe proceduri şi/sau funcţii pot forma un pachet (package). Declararea unui pachet se face în două etape:

1. Declararea specificaţiei pachetului.
2. Declararea corpului său conţinând implementarea procedurilor şi funcţiilor componente.

Specificatie

- Specificaţia unui pachet conţine declaraţiile publice aferente acestuia, vizibile pentru aplicaţiile care îl folosesc.
- Sintaxa simplificată a declarării specificaţiei este următoarea:

```
CREATE [OR REPLACE] PACKAGE nume_pachet
[AUTHID {CURRENT_USER | DEFINER}]
{IS | AS}
[definitii tipuri pt. colectii si inregistrari;
 subtipuri]
[declarare colectii, constante, exceptii, obiecte,
  inregistrari, variabile si cursori]
[declaratii functii si proceduri]
END [nume_pachet];
```

Implementare (corp)

- Specificaţia unui pachet conţine declaraţiile publice aferente acestuia, vizibile pentru aplicaţiile care îl folosesc.
- Sintaxa simplificată a declarării specificaţiei este următoarea:

```
CREATE [OR REPLACE] PACKAGE BODY nume_pachet
[AUTHID {CURRENT_USER | DEFINER}]
{IS | AS}
[definitii tipuri pentru colectii si
  inregistrari; subtipuri]
[declarare colectii, constante, exceptii,
  obiecte, inregistrari, variabile si cursori]
[declaratii functii si proceduri]
END [nume_pachet];
```

Exemplu

- Pachetul STD conţine funcţiile NUME şi PUNCTE care returnează numele şi punctajul unui student. Funcţiile au ca parametru numărul matricol al acestuia:

```
CREATE OR REPLACE PACKAGE STD AS
   FUNCTION NUME (
      PMATR NUMBER) RETURN VARCHAR2;
   FUNCTION PUNCTE (
      PMATR NUMBER) RETURN NUMBER;
END STD;
```

Exemplu

```
CREATE OR REPLACE PACKAGE BODY STD AS
  FUNCTION NUME (
    PMATR NUMBER) RETURN VARCHAR2 IS
  V_NUME VARCHAR2(10);
  BEGIN
    SELECT NUME INTO V_NUME
    FROM STUD
    WHERE MATR=PMATR;
    RETURN V_NUME;
  END NUME;
```

Exemplu

```
FUNCTION PUNCTE (
     PMATR NUMBER) RETURN NUMBER IS
   V_PUNCTE NUMBER(4);
   BEGIN
     SELECT PUNCTAJ INTO V_PUNCTE
     FROM STUD
     WHERE MATR=PMATR;
     RETURN V_PUNCTE;
   END PUNCTE;
END STD;
```

Apelare

- Folosirea subprogramelor care fac parte dintr-un pachet se face prin construcţia:

```
Pachet.subprogram([param_actuali])
```

- Exemplu: Cererea următoare returnează numele studentului, numele tutorului său şi punctajul acestuia:

```
SELECT NUME, STD.NUME(TUTOR) TUTOR,
       STD.PUNCTE(TUTOR) PUNCTE
FROM STUD;
```

- Rezultatul conţine valori nule pentru studenţii care nu au un tutor:

Apelare

NUME	TUTOR	PUNCTE
GEORGE		
VASILE	GEORGE	2890
MARIA		
ION	ALEX	1570
STANCA		
ALEX		
ELENA	STANCA	620
ADRIAN		
FLOREA		
OANA	ADRIAN	450
MARIUS	FLOREA	3230
VOICU	ADRIAN	450

Stergere pachet

- Ştergerea unui pachet se face folosind cereri DROP:

```
DROP PACKAGE BODY nume_pachet;
DROP PACKAGE nume_pachet;
```

- Exemplu:

```
DROP PACKAGE BODY STD;
DROP PACKAGE STD;
```

Declanşatori (triggers)

- Execuţia unei proceduri, funcţii sau bloc PL/SQL se poate declanşa automat la apariţia unui eveniment dacă există un declanşator (în engleză ***trigger***) asociat cu evenimentul respectiv.
- De exemplu orice modificare a punctajului unui student (folosind UPDATE) poate declanşa automat inserarea unei linii într-o tabelă de istoric unde este salvată vechea valoare a punctajului.
- Sintaxa simplificată a definirii unui declanşator este următoarea:

Declanşatori (triggers)

```
CREATE [OR REPLACE] TRIGGER nume_trigger
  {BEFORE | AFTER | INSTEAD OF}
  {DELETE | INSERT | UPDATE [OF col1 [, col2] ...]}
[OR
  {DELETE | INSERT | UPDATE [OF col3 [, col4] ...]}
] ...
ON nume_tabela
  [ [REFERENCING { OLD [AS] old [NEW [AS] new]
                 | NEW [AS] new [OLD [AS] old] } ]
[ FOR EACH ROW ]
  [WHEN (conditie)] ]
    bloc_PL_SQL
```

Descriere elemente

- **CREATE OR REPLACE** - Creaza sau inlocuieşte vechea definiţie a declanşatorului dacă acesta există deja. În cazul în care OR REPLACE lipseşte şi există deja un declanşator cu numele respectiv Oracle semnalează o eroare.
- **nume_declansator** - numele declanşatorului (identificator).
- **BEFORE** - declanşarea se face înaintea evenimentului vizat.
- **AFTER** - declanşarea se face după evenimentul vizat.
- **INSTEAD OF** - acţiunea declanşatorului înlocuieşte evenimentul vizat.
- **DELETE** - declanşarea se face la apariţia unei comenzi DELETE.
- **INSERT** - declanşarea se face la apariţia unei comenzi INSERT.
- **UPDATE...OF** - declanşarea se face la apariţia unei comenzi UPDATE care actualizează una dintre coloanele specificate (dacă sunt specificate).

Descriere elemente

- **ON** - specifică tabela careia îi este asociat declanşatorul.
- **REFERENCING** - specifică nume alternative pentru valorile vechi şi noi ale liniei. Implicit acestea sunt referite cu prefixele :old si :new.
- **FOR EACH ROW** - arată că este vorba de un declanşator la nivel de linie: acţiunea asociată se execută pentru fiecare linie afectată de evenimentul care îl declanşează.
- În lipsa acestei clauze va fi definit un declanşator la nivel de cerere (se execută o singură dată indiferent de numărul de linii afectate de evenimentul vizat).

Descriere elemente

- **WHEN** - specifică o condiţie suplimentară. Se utilizează în declanşatorii la nivel de linie şi arată că acţiunea asociată se va executa doar pentru liniile afectate care îndeplinesc condiţia WHEN.
- **bloc_PL_SQL** - acţiunea executată la declanşare.
- Dacă în zona de declaraţii blocul PL/SQL conţine:

  `PRAGMA AUTONOMOUS_TRANSACTION`

  atunci operaţiile de COMMIT şi ROLLBACK din interiorul declanşatorului sunt independente (pot fi de exemplu comise şi în cazul în care modificările care au declanşat acţiunea sunt anulate cu ROLLBACK).

Descriere elemente

- Declanşatorii se pot asocia cu:
- Cereri DML: DELETE, INSERT sau UPDATE.
- Cereri DDL: CREATE, ALTER sau DROP.
- Operatii ale bazei de date: SERVERERROR, LOGON, LOGOFF, STARTUP sau SHUTDOWN)

Exemplu

- În exemplul următor prezentăm doi declanşatori

1. unul la nivel de linie care salvează într-o tabelă ISTORIC valorile punctajelor studenţilor atunci când coloana respectivă este actualizată:

```
CREATE OR REPLACE TRIGGER scrie_istorie
  AFTER UPDATE OF punctaj ON stud
  FOR EACH ROW
BEGIN
    INSERT INTO ISTORIC
      VALUES (:old.MATR, :old.punctaj,
  :new.punctaj);
END;
```

Exemplu

- În exemplul următor prezentăm doi declanşatori:

2. unul la nivel de cerere care înregistrează cine a modificat tabela SPEC şi când (dar nu şi cum):

```
CREATE OR REPLACE TRIGGER cine_modifica
   BEFORE INSERT OR DELETE OR UPDATE ON spec
   BEGIN
     INSERT INTO MODIF VALUES (USER,
  SYSDATE);
   END;
```

Exemplu

- Efectul unei actualizări a tabelei STUD asupra conţinutului tabelei ISTORIC este următorul:

```
UPDATE STUD
SET PUNCTAJ = PUNCTAJ + 100
WHERE CODS = 11;

SELECT * FROM ISTORIC;

 MATR PVECHI  PNOU
----- ------ -----
 1456   2890  2990
 1325    390   490
 1645   1400  1500
```

Alt exemplu

- Se poate folosi un declansator (trigger) pentru generarea unei chei surogat folosind o secventa:

```
CREATE SEQUENCE SECV;
--
CREATE TABLE TEST
  (PK NUMBER PRIMARY KEY,
  NUME VARCHAR2(40) NOT NULL,
  MEDIE NUMBER);
```

Alt exemplu

```
CREATE OR REPLACE TRIGGER NOU_PK
  BEFORE INSERT ON TEST
     FOR EACH ROW
  BEGIN
     IF :NEW.EMPNO IS NULL THEN
       :NEW.EMPNO := SECV.NEXTVAL;
     END IF;
END;
--
```

Alt exemplu

```
INSERT INTO TEST (NUME, MEDIE) VALUES ('ION', 10.00);
INSERT INTO TEST (NUME, MEDIE) VALUES ('DAN', 9.90);
INSERT INTO TEST VALUES (7, 'MARIA', 10.00);
COMMIT;
```

- Rezultat: inserarea a 3 linii cu cheile 1, 2 si 7 (de retinut ca secventa SECV va genera mai tarziu valoarea 7!).

:new si :old

- :new si :old se pot folosi astfel:

	:NEW	:OLD
INSERT	√	N/A
UPDATE	√	√
DELETE	N/A	√

Stergere

- Ştergerea unui declanşator se face cu DROP TRIGGER. Exemplu:

```
DROP TRIGGER scrie_istorie;
```

- Observaţie: în cazul în care la crearea unui obiect de tip procedură, funcţie, pachet sau declanşator se semnalează erori acestea pot fi vizualizate cu comanda:

```
SHOW ERRORS;
```

SFARSIT